

Laporan Tugas Besar
EL2008 Pemecahan Masalah dengan Pemrograman



Sistem Inventarisasi Laboratorium dengan Sistem Database
Berbasis Mikrokontroler Arduino UNO
K03 - Kelompok 6

Disusun Oleh:

Muhammad Ammar Hanif 13224087

Maria Vanessa Soritan 13224078

Hufnagel Ruben Lenard S. 13224015

Christian Jonathan Hutajulu 13224108

Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

Daftar Isi

Daftar Isi.....	1
Daftar Tabel.....	2
Daftar Gambar.....	3
I. Deskripsi Permasalahan.....	5
II. Dekomposisi dan Solusi Permasalahan.....	5
2.1 Dekomposisi Masalah.....	5
2.2 Solusi Masalah.....	8
III. Flowchart dan Data Flow Diagram (DFD).....	13
3.1.1 Flowchart app_logic.c.....	13
3.1.2 Flowchart database.c.....	21
3.1.3 Flowchart serial_comm.c.....	34
3.2 Data Flow Diagram (DFD).....	41
3.2.1 DFD Arduino UNO System.....	42
3.2.2 DFD Client Interface System.....	43
IV. Penjelasan Logika dan Penjelasan Penggunaan Struktur Data.....	45
4.1 Struktur Penggunaan Memori Pada Arduino.....	45
4.2 Struktur Data Pada Memori Volatile.....	46
4.3 Struktur Data Pada Memori Non-Volatile.....	47
4.4 Logika Kompresi dan Pack/Unpack Data Pada Memori Non-Volatile.....	48
4.5 Mekanisme Konsistensi Data Dari Input.....	51
V. Contoh Studi Kasus (Input-Output) Untuk Masing-Masing Proses dan Analisis Hasil dan Justifikasi Pendekatan Implementasi.....	51
VI. Pembagian Kerja Kelompok Secara Substansial Berdasarkan Dekomposisi Permasalahan.....	78
6.1 Pembagian Tugas Pengerjaan Laporan dan Slide Presentasi.....	78
6.2 Pembagian Tugas Pengerjaan Flowchart.....	79
6.3 Pembagian Tugas Pembuatan Program C.....	81
VII. Analisis Keterbatasan Sistem.....	83
7.1 Keterbatasan Kapasitas Memori Non-Volatile.....	83
7.2 Restriksi Format Karakter dan Pemotongan String.....	84
7.3 Batas Kuantitas Barang dan Rentang ID.....	84
7.4 Delay Pada Operasi I/O.....	84
7.5 Ketergantungan Terhadap Proses Validasi Client Interface (PC).....	85
VIII. Kesimpulan dan Saran.....	85
8.1 Kesimpulan.....	85
8.2 Saran.....	86
IX. Sumber.....	86
X. Lampiran.....	86

Daftar Tabel

Tabel 2.1.1 Rincian Data Sistem Inventarisasi Laboratorium.....	5
Tabel 2.1.2 Rincian Data Sistem Inventarisasi Laboratorium.....	6
Tabel 2.1.3 Rincian Sistem Inventarisasi Laboratorium.....	7
Tabel 2.1.4 Rincian Program Sistem Database Arduino.....	8
Tabel 2.1.5 Rincian Program Sistem Client Interface.....	8
Tabel 2.2.1 Rincian Function Program Sistem Database Arduino.....	8
Tabel 2.2.2 Rincian Function Program Sistem Client Interface.....	10
Tabel 4.1 Susunan Alokasi Bit pada Struktur ItemData.....	47
Tabel 4.2. Pemetaan Kampus Kompresi Alfanyumerik 6-Bit.....	48
Tabel 4.3. Pemetaan Kampus Kompresi 5-Bit (variabel PIC).....	49
Tabel 5.1 Studi Kasus serta Analisis dan Justifikasinya.....	51
Tabel 6.1.1 Pembagian Tugas Pengerjaan Laporan dan Slide Presentasi.....	78
Tabel 6.1.2 Pembagian Tugas Pengerjaan Flowchart.....	79
Tabel 6.1.3 Pembagian Tugas Pembuatan Program C.....	82

Daftar Gambar

Gambar 3.1.1 Flowchart Fungsi runApp.....	13
Gambar 3.1.2 Flowchart Fungsi addInventory.....	15
Gambar 3.1.3 Flowchart Fungsi deleteInventory.....	16
Gambar 3.1.4 Flowchart Fungsi searchByID.....	17
Gambar 3.1.5 Flowchart Fungsi updateStockAndStatus.....	17
Gambar 3.1.6 Flowchart Fungsi displayAllInventory.....	19
Gambar 3.1.7 Flowchart Fungsi displaySummary.....	20
Gambar 3.1.1.8 Flowchart Fungsi fetchSerialUntil.....	20
Gambar 3.1.2.1 Flowchart Fungsi initUART.....	21
Gambar 3.1.2.2 Flowchart printString.....	21
Gambar 3.1.2.3 Flowchart Fungsi initDatabase.....	22
Gambar 3.1.2.4 Flowchart Fungsi processSerialInput.....	22
Gambar 3.1.2.5 Flowchart Fungsi encodeChar6Bit.....	23
Gambar 3.1.2.6 Flowchart Fungsi decodeChar6Bit.....	23
Gambar 3.1.2.7 Flowchart Fungsi encodeChar5Bit.....	24
Gambar 3.1.2.8 Flowchart Fungsi decodeChar5Bit.....	24
Gambar 3.1.2.9 Flowchart Fungsi packItemData.....	25
Gambar 3.1.2.10 Flowchart Fungsi unpackItemName.....	26
Gambar 3.1.2.11 Flowchart Fungsi unpackItemPIC.....	26
Gambar 3.1.2.12 Flowchart Fungsi insertItem.....	27
Gambar 3.1.2.13 Flowchart Fungsi deleteItem.....	28
Gambar 3.1.2.14 Flowchart Fungsi updateItem.....	29
Gambar 3.1.2.15 Flowchart Fungsi syncListToEEPROM.....	30
Gambar 3.1.2.16 Flowchart Fungsi printInventory.....	30
Gambar 3.1.2.17 Flowchart Fungsi parseAndAddCommand.....	31
Gambar 3.1.2.18 Flowchart Fungsi dataAvailableUART.....	32
Gambar 3.1.2.19 Flowchart Fungsi receiveUART.....	32
Gambar 3.1.2.20 Flowchart Fungsi transmitUART.....	32
Gambar 3.1.2.21 Flowchart Fungsi printInt.....	33
Gambar 3.1.3.1. Flowchart Fungsi initSerialPort.....	35
Gambar 3.1.3.2. Flowchart Fungsi sendSerialData.....	36
Gambar 3.1.3.3. Flowchart Fungsi receiveSerialData.....	37
Gambar 3.1.3.4. Flowchart Fungsi closeSerialPort.....	38
Gambar 3.1.4.1 Flowchart Fungsi printMainMenu.....	39
Gambar 3.1.4.2 Flowchart Fungsi getUserChoice.....	39
Gambar 3.1.4.3 Flowchart Fungsi promptString.....	40
Gambar 3.1.4.4 Flowchart Fungsi promptInt.....	40
Gambar 3.1.4.5 Flowchart Fungsi printTableHeader.....	41
Gambar 3.1.4.6 Flowchart Fungsi printTableFooter.....	41
Gambar 3.2.1.1. DFD Level 0 Arduino UNO System.....	42

Gambar 3.2.1.2. DFD Level 1 Arduino UNO System.....	42
Gambar 3.2.1.3. DFD Level 2 Arduino UNO System.....	43
Gambar 3.2.2.1. DFD Level 0 Client Interface System.....	43
Gambar 3.2.2.2. DFD Level 1 Client Interface System.....	44
Gambar 3.2.2.3. DFD Level 2 Client Interface System.....	45
Gambar 4.1. Diagram Blok Umum Antara SRAM dan EEPROM.....	46
Gambar 4.2. Struktur Elemen Singly Linked List.....	47
Gambar 4.3. Deklarasi atribut pada database.h.....	48
Gambar 4.4. Ilustrasi Operasi Bitwise Packing Variabel PIC.....	50
Gambar 4.5. Ilustrasi Operasi Bitmasking saat proses Unpacking.....	51

I. Deskripsi Permasalahan

Laboratorium Embedded Systems punya banyak komponen dan peralatan yang dibutuhkan untuk berbagai kegiatan, layaknya praktikum, penelitian, dan proyek mahasiswa seperti sensor, Arduino, ESP32, Raspberry Pi, modul komunikasi, driver motor, multimeter, kabel, dan perangkat pendukung lainnya. Pendataan inventaris barang masih dilakukan secara manual selama ini, maka ingin dibuat sistem inventaris digital sederhana untuk menggantikan pencatatan manual.

Sistem inventaris digital ini berbasis Arduino Uno, dimana setiap data dalam inventaris bisa dicari berdasarkan ID barangnya. Untuk database keseluruhannya memaksimalkan memory dari Arduino Uno. Tiap data inventaris akan memuat ID barang, nama komponen/peralatan, kategori barang, jumlah stok, lokasi penyimpanan, status barang, pemilik barang, serta PIC untuk barang. Database memanfaatkan Arduino Uno dengan operasi dasar inventaris diakses dari Serial Monitor dan terdiri dari operasi menambah data, menghapus data, mencari data berdasarkan ID barang, memperbarui stok, memperbarui status dan menampilkan seluruh data, dan menampilkan ringkasan inventaris. Sistem akan menangani kasus permasalahan seperti ID duplikat, ID tidak ditemukan, stok tidak mencukupi, data kosong, kapasitas memori hampir habis, penghapusan node pertama, tengah, dan akhir linked list. Cara kerja program adalah pemanggilan fungsi void dari main.

II. Dekomposisi dan Solusi Permasalahan

2.1 Dekomposisi Masalah

Berdasarkan deskripsi permasalahan, sistem inventarisasi laboratorium membutuhkan penyimpanan data dengan rincian data sebagai berikut:

Tabel 2.1.1 Rincian Data Sistem Inventarisasi Laboratorium.

No	Variabel Data	Keterangan
1	ID	Identifier unik tiap barang
2	Nama Item	Nama komponen
3	Kategori	Microcontroller / Aktuator / Sensor / Lainnya
4	Lokasi (Nomor Rak)	Posisi rak penyimpanan barang
5	Jumlah Tersedia	Stok barang yang tersedia untuk

		dipinjam
6	Jumlah Dipinjam	Jumlah unit barang yang sedang dipinjam
7	Jumlah Rusak	Jumlah unit barang dalam kondisi rusak
8	Pemilik Barang	(kami asumsikan Laboratorium Embedded Systems merupakan pemilik semua barang)
9	PIC Barang	Penanggung jawab barang

Variabel data berikut akan menjadi dasar untuk struktur data yang digunakan oleh seluruh sistem.

Dalam rangka memaksimalkan sistem database kami, diperlukan sistem database yang tidak hilang saat Arduino UNO dimatikan, di-*restart*, atau diputus dari komputer. Sistem database membutuhkan media penyimpanan yang bersifat *non-volatile*, yaitu memori yang tetap menyimpan data meskipun sumber catu daya/media diputus. Untuk memenuhi kebutuhan ini, digunakan EEPROM internal Arduino UNO dengan kapasitas total 1 KB (1024 byte) per perangkat. Pemilihan penggunaan EEPROM mengindikasikan bahwa kapasitas data pada sistem akan sangat terbatas, sehingga efisiensi ukuran data per item akan menjadi pertimbangan utama kami dalam perancangan struktur data.

Untuk memaksimalkan jumlah item yang bisa ditampung dalam 1 KB EEPROM, kami melakukan 2 langkah optimasi, yaitu:

1. Penghapusan atribut yang tidak esensial
Pada data, data pemilik barang tidak disimpan dengan asumsi bahwa seluruh barang dalam sistem inventarisasi laboratorium dimiliki oleh Laboratorium Embedded Systems itu sendiri. Sistem juga tidak menggunakan QR code untuk ID identifier karena sistem berfokus pada perancangan sistem database itu sendiri. Walau tidak menggunakan QR Code, sebuah *integer* ID tetap menjadi *key* untuk mengakses data barang berkaitan sesuai dengan permintaan kebutuhan sistem.
2. Pembatasan rentang nilai dan tipe data (*bit-width constraint*)
Tiap data akan diberi batasan nilai yang ketat agar sesuai dengan kebutuhan realistis operasional laboratorium. Tiap data dapat direpresentasikan dengan jumlah bit minimum sebagai berikut:

Tabel 2.1.2 Rincian Data Sistem Inventarisasi Laboratorium.

Atribut Data	Rentang/Format Data	Representasi Data (bit)
ID	0-127	int 7-bit
Nama Item	Maks. 8 karakter (A-Z, a-z, 0-9)	8 × custom 6-bit char (48-bit / 6 byte total)
Kategori	0-3 (0: Microcontroller, 1: Aktuator, 2: Sensor, 3: Lainnya)	int 2-bit
Jumlah Tersedia	0-63	int 6-bit
Jumlah Dipinjam	0-63	int 6-bit
Jumlah Rusak	0-63	int 6-bit
Pemilik Barang	-	Tidak disimpan
PIC Barang	Tepat 3 karakter (A-Z saja)	3 × custom 5-bit char (15-bit, dikemas dalam 1 variabel 16-bit)

Dengan aturan pembatasan ini, tiap data barang dapat di-*packed* ke jumlah byte yang lebih kecil dibanding menggunakan tipe data standar seperti integer dan char[]. Oleh karena itu, kapasitas EEPROM 1 KB dapat menampung jumlah item secara maksimal dan efisien.

Arduino UNO juga memiliki keterbatasan bukan hanya memori penyimpanan, namun juga pada kemampuan pemrosesan dan tampilan. Maka dari itu, kami memecah program kami menjadi 2 bagian besar sistem, yaitu:

Tabel 2.1.3 Rincian Sistem Inventarisasi Laboratorium.

Sistem Database Arduino UNO	Sistem Client Interface (<i>terminal</i>)
Sistem database arduino pada sistem adalah <i>avr_mcu</i> akan melakukan proses: • Pengolahan dan penyimpanan data pada operasi di EEPROM • Penerimaan request dan pengiriman response melalui komunikasi serial • Validasi dan penanganan error pada level data, seperti kasis ID duplikat, ID tidak ditemukan, stok tidak cukup, memori hampir penuh, dan linked list penuh.	Sistem <i>client_interface</i> akan melakukan proses: • Tampilan menu dan interaksi dengan pengguna • Validasi input pada level interface sebelum dikirim ke Arduino • Pengiriman hasil operasi dasar ke <i>avr_mcu</i> melalui komunikasi serial, juga menampilkan response dari Arduino.

Pemisahan ini dilakukan dengan tujuan Arduino UNO dapat fokus sepenuhnya untuk perannya sebagai sistem penyimpanan data dengan beban komputasi minimal, sementara seluruh logika interaksi yang kompleks ditangani oleh komputer yang memiliki sumber daya jauh lebih besar.

Pada masing-masing sistem, tiap sistem disusun kembali secara modular, terstruktur, dan mudah diuji secara terpisah, program dibagi menjadi beberapa file .c dan .h dengan rincian sebagai berikut:

Tabel 2.1.4 Rincian Program Sistem Database Arduino.

File	Keterangan
<i>database.h & database.c</i>	Membaca/tulis EEPROM layaknya proses <i>bit-packing/unpacking</i> , serta berisi beberapa logika inventaris, serta menjalankan proses komunikasi
<i>main.c</i>	Menginisialisasi UART dan database, serta memproses serial input.
<i>Makefile</i>	Skrip untuk kompilasi dan upload program ke Arduino UNO.

Tabel 2.1.5 Rincian Program Sistem Client Interface.

File	Keterangan
<i>main.c</i>	Memanggil inisialisasi dan menu utama
<i>app_logic.c & app_logic.h</i>	Logika utama sistem untuk operasi dasar.
<i>user_interface.c & user_interface.h</i>	Membuat tampilan interface kepada user
<i>serial_comm.c & serial_comm.h</i>	Mengatur komunikasi serial data antara PC dengan Arduino
<i>Makefile</i>	Skrip untuk kompilasi program di PC (<i>linux base</i>)
<i>build.bat</i>	Skrip untuk kompilasi program di PC (<i>windows base</i>)

2.2 Solusi Masalah

Dalam menyelesaikan masalah sistem inventarisasi ini, maka diimplementasikan fungsi-fungsi pada program C sebagaimana tiap fungsi membantu menjawab kebutuhan sistem. Seluruh fungsi bertipe void dan fungsi main hanya memanggil void.

Tabel 2.2.1 Rincian Function Program Sistem Database Arduino.

File	Fungsi	Keterangan
database.h & database.c	<code>void initUART(void);</code>	Fungsi <i>initUart</i> berfungsi untuk menginisialisasi register <i>hardware</i> untuk komunikasi serial pada Baudrate 9600.
	<code>void printString(const char* str);</code>	Fungsi <i>printString</i> berfungsi untuk mengirimkan sebuah <i>string</i> karakter demi karakter melalui komunikasi UART hingga mencapai karakter akhir string (null terminator).
	<code>void initDatabase(void);</code>	Fungsi <i>initDatabase</i> berfungsi untuk membaca data fisik dari EEPROM saat mikrokontroler menyala dan merekonstruksi <i>Singly Linked List</i> ke dalam SRAM.
	<code>void processSerialInput(void);</code>	Fungsi <i>processSerialInput</i> berfungsi untuk membaca data masuk dari UART karakter demi karakter ke dalam buffer statis hingga mendeteksi karakter <i>newline</i> (<code>'\n'</code>), lalu mem-parsing teks tersebut untuk mengeksekusi perintah inventaris yang sesuai.
	<code>void encodeChar6Bit(char inChar, uint8_t* outByte);</code>	Fungsi <i>encodeChar6Bit</i> berfungsi untuk melakukan proses <i>encoding</i> karakter ASCII standar [A-Z, a-z, 0-9] menjadi representasi memori 6-bit untuk tujuan efisiensi.
	<code>void decodeChar6Bit(uint8_t inByte, char* outChar);</code>	Fungsi <i>decodeChar6Bit</i> berfungsi untuk melakukan proses <i>decoding</i> nilai 6-bit kembali menjadi karakter ASCII standar 8-bit.
	<code>void encodeChar5Bit(char inChar, uint8_t* outByte);</code>	Fungsi <i>encodeChar5Bit</i> berfungsi untuk melakukan proses <i>encoding</i> huruf kapital ASCII [A-Z] menjadi representasi memori 5-bit untuk tujuan efisiensi.
	<code>void decodeChar5Bit(uint8_t inByte, char* outChar);</code>	Fungsi <i>decodeChar5Bit</i> berfungsi untuk melakukan proses <i>decoding</i> nilai 5-bit kembali menjadi huruf kapital ASCII 8-bit.
	<code>void packItemData(ItemData* item, uint8_t id, const char* name, uint8_t cat, uint8_t loc, uint8_t qTer, uint8_t qDip, uint8_t qRus, const char* pic);</code>	Fungsi <i>packItemData</i> berfungsi untuk melakukan kompresi dan menyusun seluruh variabel parameter menjadi satu <i>struct</i> berukuran 12-byte.
	<code>void unpackItemName(ItemData* item, char* outBuffer);</code>	Fungsi <i>unpackItemName</i> berfungsi untuk melakukan dekompresi dan menyusun 8 potong nilai 6-bit dari <i>struct</i> menjadi sebuah <i>string</i> utuh (Nama Item).

	<pre>void unpackItemPIC (ItemData* item, char* outBuffer);</pre>	Fungsi <i>unpackItemPIC</i> berfungsi untuk melakukan ekstraksi gabungan <i>bitwise</i> 16-bit menjadi <i>string</i> utuh berisi 3 karakter (PIC Barang).
	<pre>void insertItem (ItemData newData);</pre>	Fungsi <i>insertItem</i> berfungsi untuk menambahkan data baru ke posisi paling belakang pada <i>linked list</i> . Fungsi ini bekerja dengan mengalokasikan memori untuk <i>node</i> baru terlebih dahulu.
	<pre>void deleteItem (uint8_t targetId);</pre>	Fungsi <i>deleteItem</i> berfungsi untuk menghapus sebuah data (<i>node</i>) dari <i>linked list</i> berdasarkan ID target yang dicari. Fungsi ini akan mencari node ID yang dimaksud dan dilepas dari <i>list</i> , lalu memorinya dibebaskan.
	<pre>void updateItem (uint8_t targetId, uint8_t qTer, uint8_t qDip, uint8_t qRus);</pre>	Fungsi <i>updateItem</i> berfungsi untuk memperbarui data status (stok tersedia, dipinjam, dan rusak) pada <i>node</i> yang memiliki ID tertentu di dalam <i>linked list</i> . Jika data ditemukan, nilai baru akan disimpan, dan disinkronisasikan ke EEPROM.
	<pre>void syncListToEEPROM (void);</pre>	Fungsi <i>syncListToEEPROM</i> berfungsi untuk menyinkronkan dan menimpa status terbaru <i>Linked List</i> dari SRAM ke dalam penyimpanan fisik EEPROM.
	<pre>void printInventory (void);</pre>	Fungsi <i>printInventory</i> berfungsi untuk mengirimkan seluruh data inventaris di dalam <i>linked list</i> karakter demi karakter melalui UART..
	<pre>void parseAndAddCommand (ch ar* commandString);</pre>	Fungsi <i>parseAndAddCommand</i> berfungsi untuk memotong <i>string</i> menggunakan perintah strtok, mengekstrak parameter barang, lalu membungkusnya ke dalam struktur <i>ItemData</i> .
	<pre>void dataAvailableUART (uin t8_t* available)</pre>	Fungsi <i>dataAvailableUART</i> berfungsi untuk memeriksa flag <i>RXC0</i> pada register <i>UCSR0A</i> untuk mendeteksi ketersediaan data masuk dari PC.
	<pre>void receiveUART (char* receivedData)</pre>	Fungsi <i>receiveUART</i> berfungsi untuk menangkap dan mengembalikan byte data mentah secara langsung dari register penerima UDR0.
	<pre>void transmitUART (char data)</pre>	Fungsi <i>transmitUART</i> berfungsi untuk menahan eksekusi program sementara hingga buffer pengiriman perangkat keras siap (<i>UCSR0A</i>), lalu mengirimkan satu karakter mentah (byte) secara langsung melalui register transmisi <i>UDR0</i> .
<i>main.c</i>	-	

Tabel 2.2.2 Rincian Function Program Sistem Client Interface.

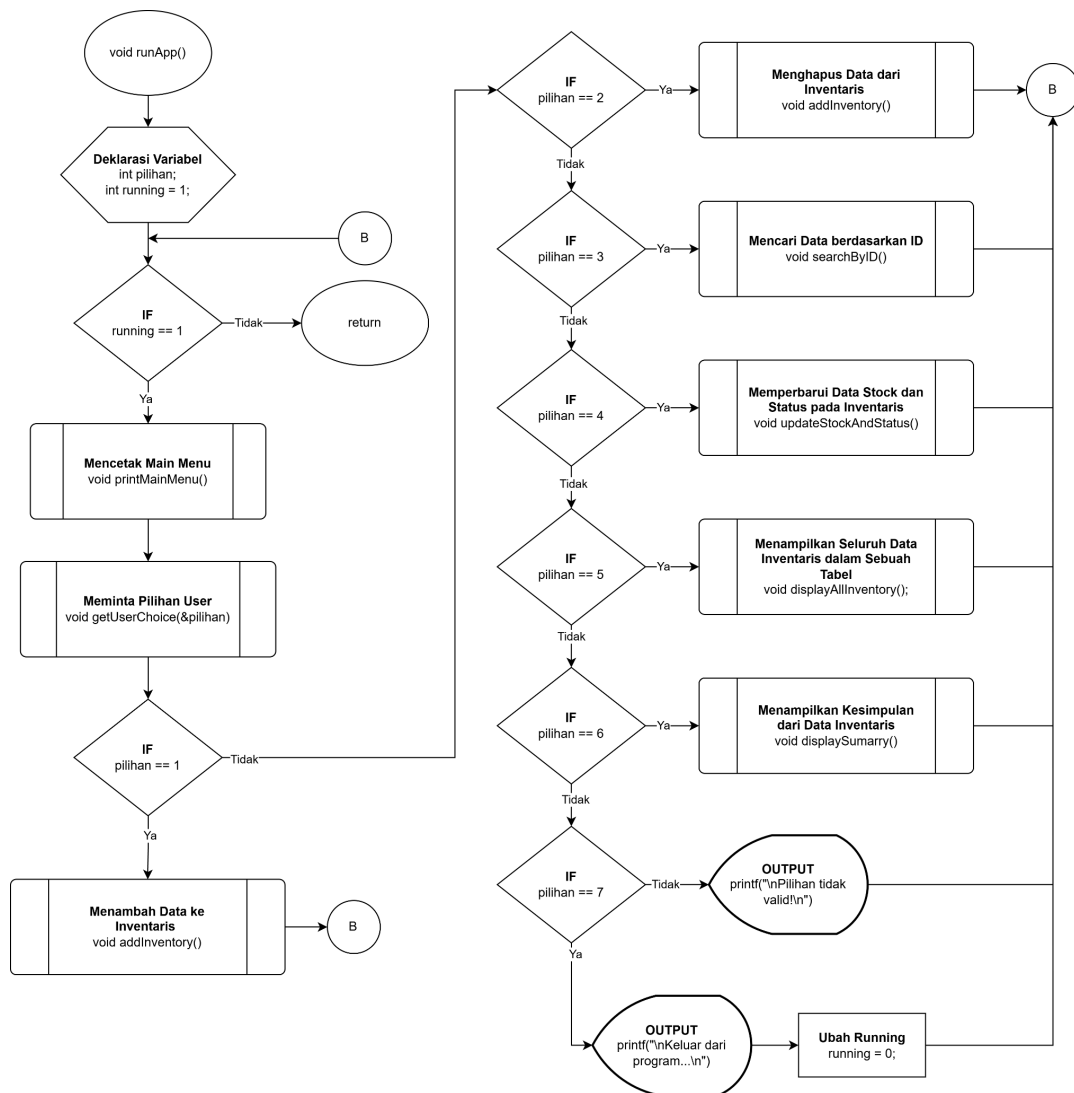
File	Fungsi	Keterangan
<i>main.c</i>	-	
<i>app_logic.c & app_logic.h</i>	<code>void runApp()</code>	Fungsi <i>runApp</i> menampilkan antarmuka menu utama kepada pengguna, menerima input pilihan angka, lalu mengarahkan eksekusi ke fungsi-fungsi fitur inventaris yang sesuai secara berulang hingga program dihentikan.
	<code>void addInventory()</code>	Fungsi <i>addInventory</i> menerima parameter detail data baru dari pengguna, memverifikasi agar ID tidak duplikat via komunikasi serial, lalu mengirimkan perintah ADD untuk mendaftarkan data tersebut jika seluruh validasi berhasil.
	<code>void deleteInventory()</code>	Fungsi <i>deleteInventory</i> menerima input ID dari pengguna, memverifikasi keberadaannya via komunikasi serial, lalu mengirimkan perintah DEL untuk menghapus data tersebut jika ID tersebut valid.
	<code>void searchByID()</code>	Fungsi <i>searchByID</i> menerima input ID dari pengguna, mengambil seluruh data inventaris via komunikasi serial menggunakan perintah GET_ALL, lalu mem-parsing dan menampilkan data yang sesuai dengan ID tersebut di layar.
	<code>void updateStockAndStatus()</code>	Fungsi <i>updateStockAndStatus</i> menerima input ID dari pengguna, memverifikasi keberadaannya via komunikasi serial, lalu mengirimkan perintah UPDATE berisi data stok baru jika ID tersebut valid.
	<code>void displayAllInventory()</code>	Fungsi <i>displayAllInventory</i> mengambil seluruh data inventaris via komunikasi serial menggunakan perintah GET_ALL, lalu mem-parsing dan menampilkan data tersebut ke dalam bentuk tabel di layar.
	<code>void displaySummary()</code>	Fungsi <i>displaySummary</i> mengambil data barang dengan perintah GET_ALL, lalu menghitung dan menampilkan total jenis barang beserta jumlah seluruh stok (tersedia, dipinjam, dan rusak) dalam bentuk ringkasan.
	<code>Void fetchSerialUnit()</code>	Fungsi <i>fetchSerialUntil</i> berfungsi untuk membaca dan mengumpulkan data dari komunikasi serial secara berkala ke dalam <i>serialBuffer</i> hingga kata kunci target ditemukan, dengan batas waktu tunggu maksimal 3 detik.
<i>user_interface.</i>	<code>void printMainMenu()</code>	Menampilkan menu utama sistem inventaris

<i>c & user_interface.h</i>		yang berisi daftar operasi yang tersedia bagi pengguna
	<code>void getUserChoice(int* choice)</code>	Menerima pilihan menu dari pengguna dan menyimpannya ke variabel yang dituju
	<code>void promptString(const char* promptText, char* inputBuffer, int maxLength)</code>	Menampilkan prompt & menerima input berupa string dengan panjang maksimum tertentu
	<code>void promptInt(const char* promptText, int* inputValue)</code>	Menampilkan prompt dan menerima input berupa bilangan bulat dari pengguna
	<code>void printTableHeader()</code>	Menampilkan header tabel inventaris
	<code>void printTableFooter()</code>	Menampilkan footer tabel inventaris
<i>serial_comm.c & serial_comm.h</i>	<code>void initSerialPort(const char* portName, int* statusFlag);</code>	Fungsi <i>initSerialPort</i> berfungsi untuk membuka koneksi POSIX ke port serial yang terhubung dengan Arduino dan mengkonfigurasi parameter (<i>baudrate, parity, stop bit</i>).
	<code>void sendSerialData(const char* dataStr);</code>	Fungsi <i>sendSerialData</i> berfungsi untuk mengirimkan string perintah secara utuh (seperti "GET_ALL") dari PC menuju mikrokontroler.
	<code>void receiveSerialData(char* buffer);</code>	Fungsi <i>receiveSerialData</i> berfungsi untuk membaca aliran byte asinkron yang datang dari kabel USB dan menyimpannya ke dalam buffer karakter untuk dievaluasi lebih lanjut.
	<code>void closeSerialPort();</code>	Fungsi <i>closeSerialPort</i> berfungsi untuk menutup jalur komunikasi secara aman (<i>safe exit</i>) saat program antarmuka diakhiri untuk mencegah status port terkunci oleh sistem operasi.

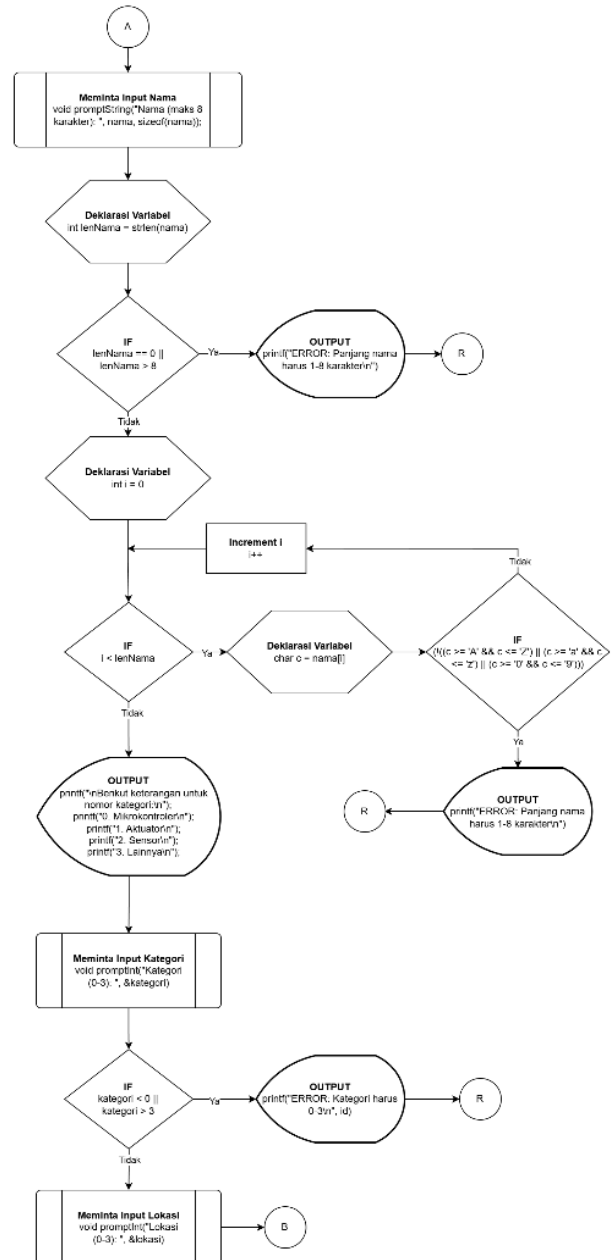
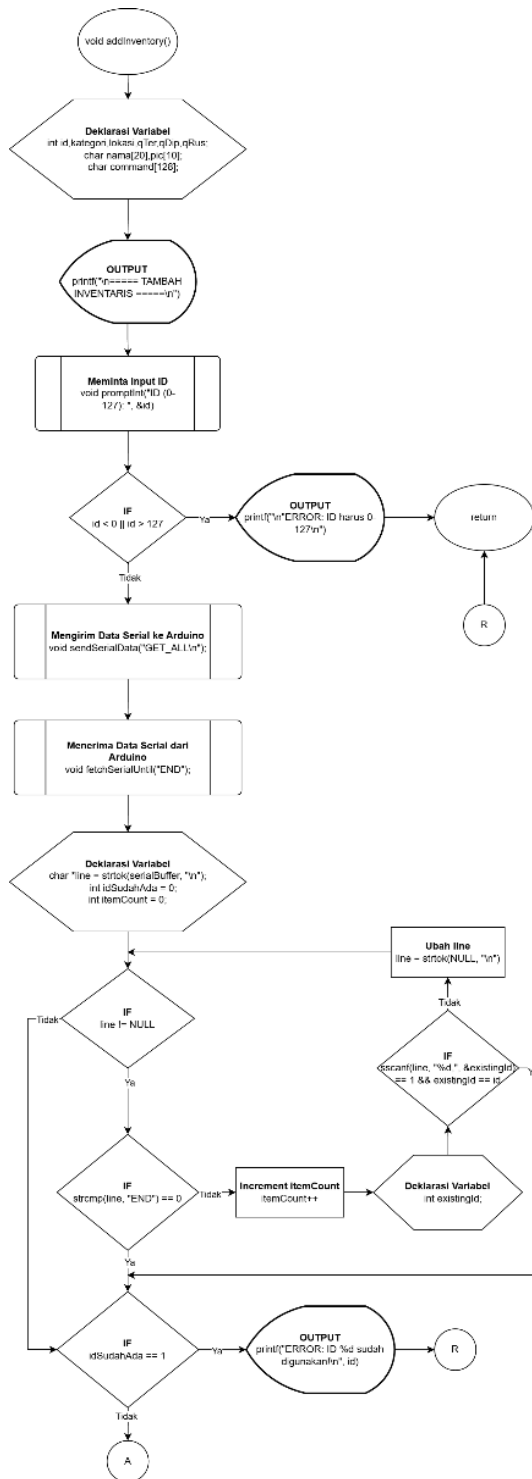
III. Flowchart dan Data Flow Diagram (DFD)

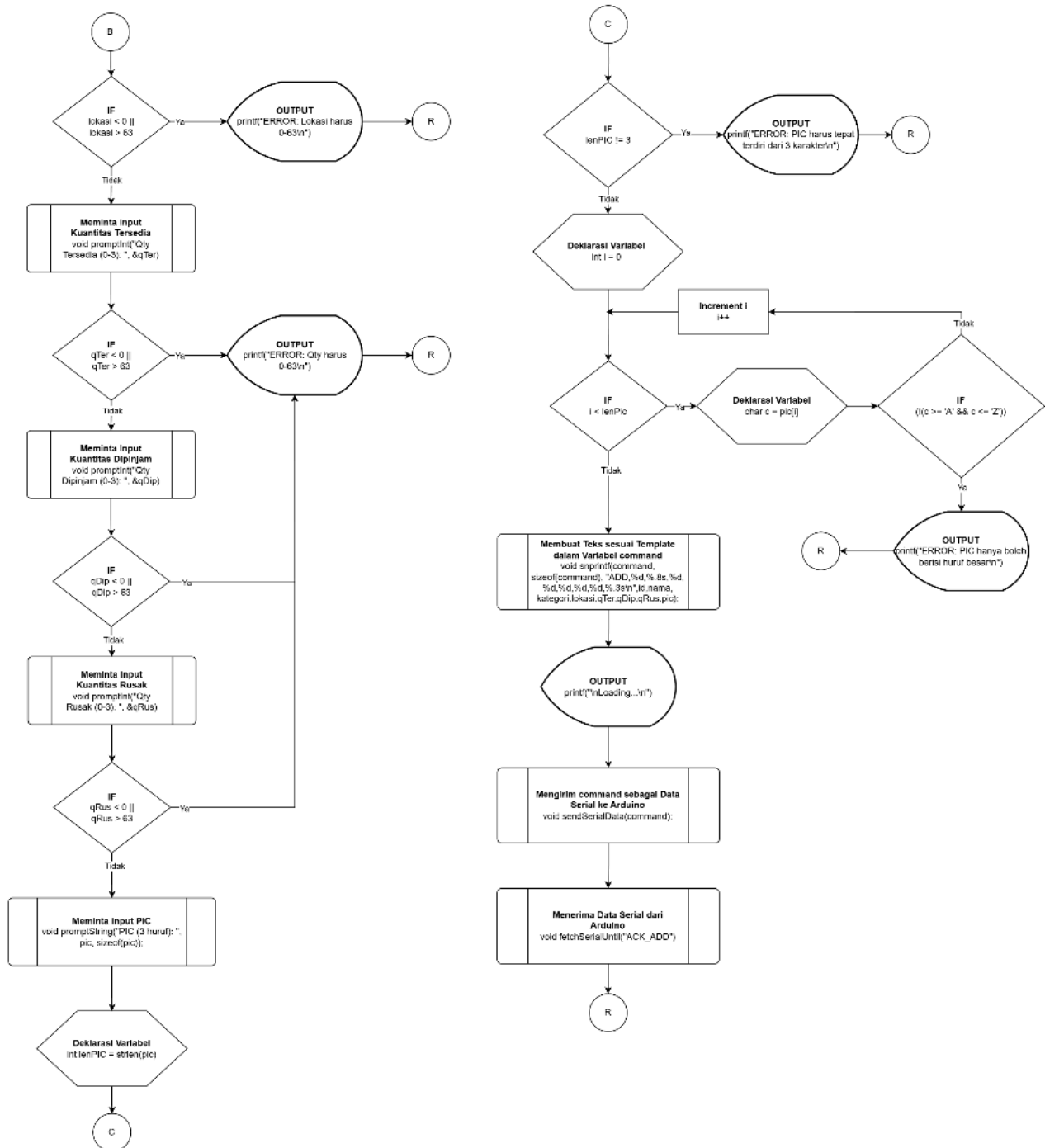
3.1 Flowchart

3.1.1 Flowchart app_logic.c

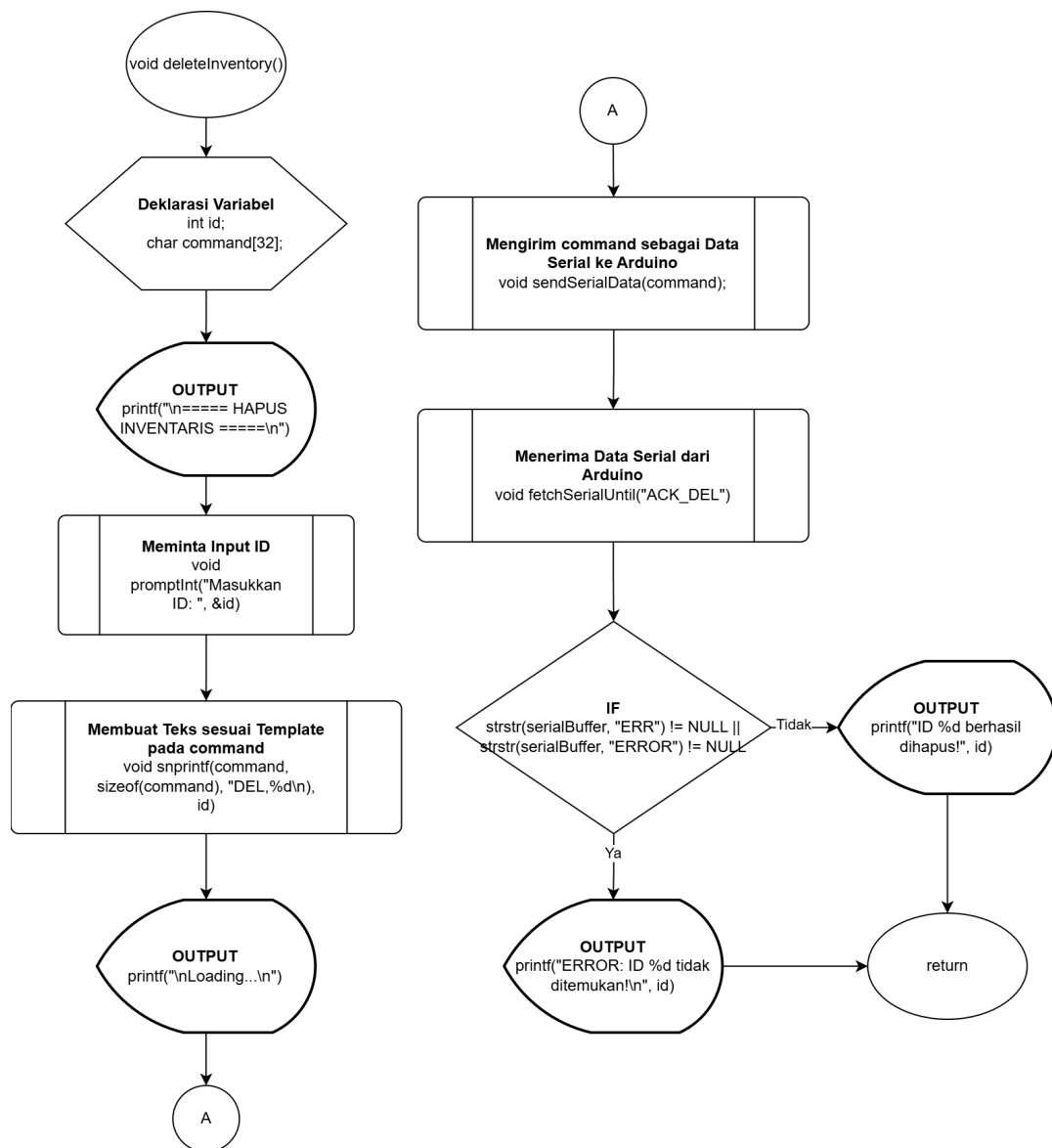


Gambar 3.1.1 Flowchart Fungsi *runApp*.

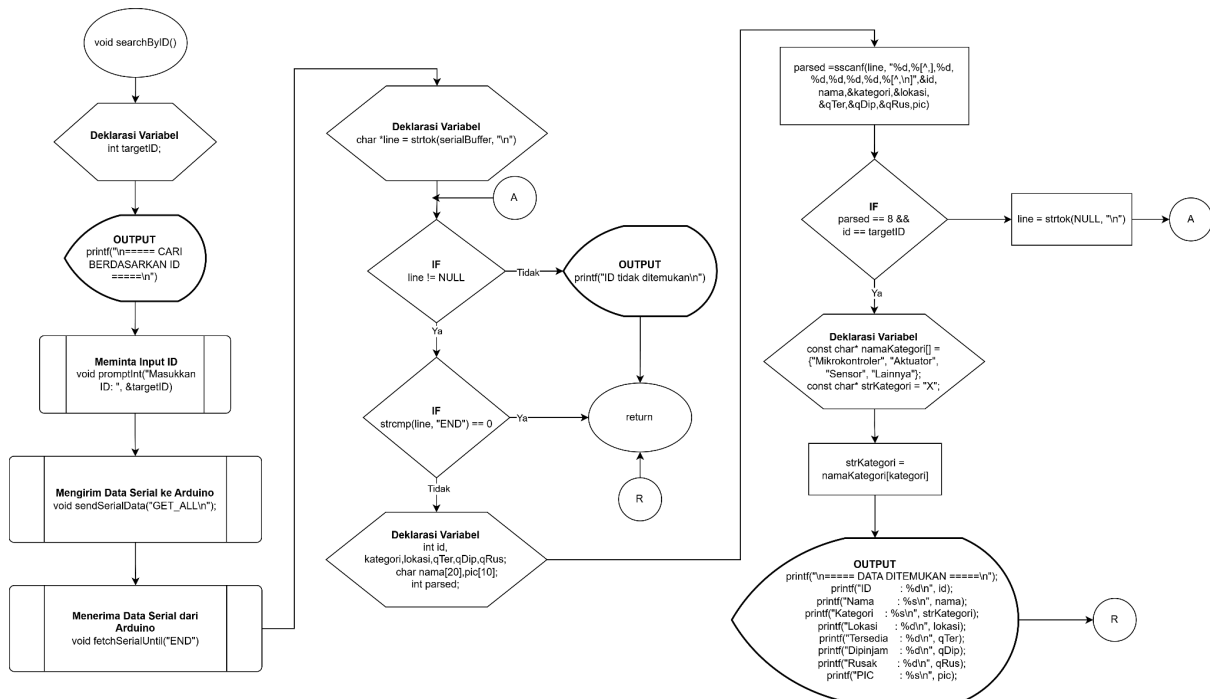




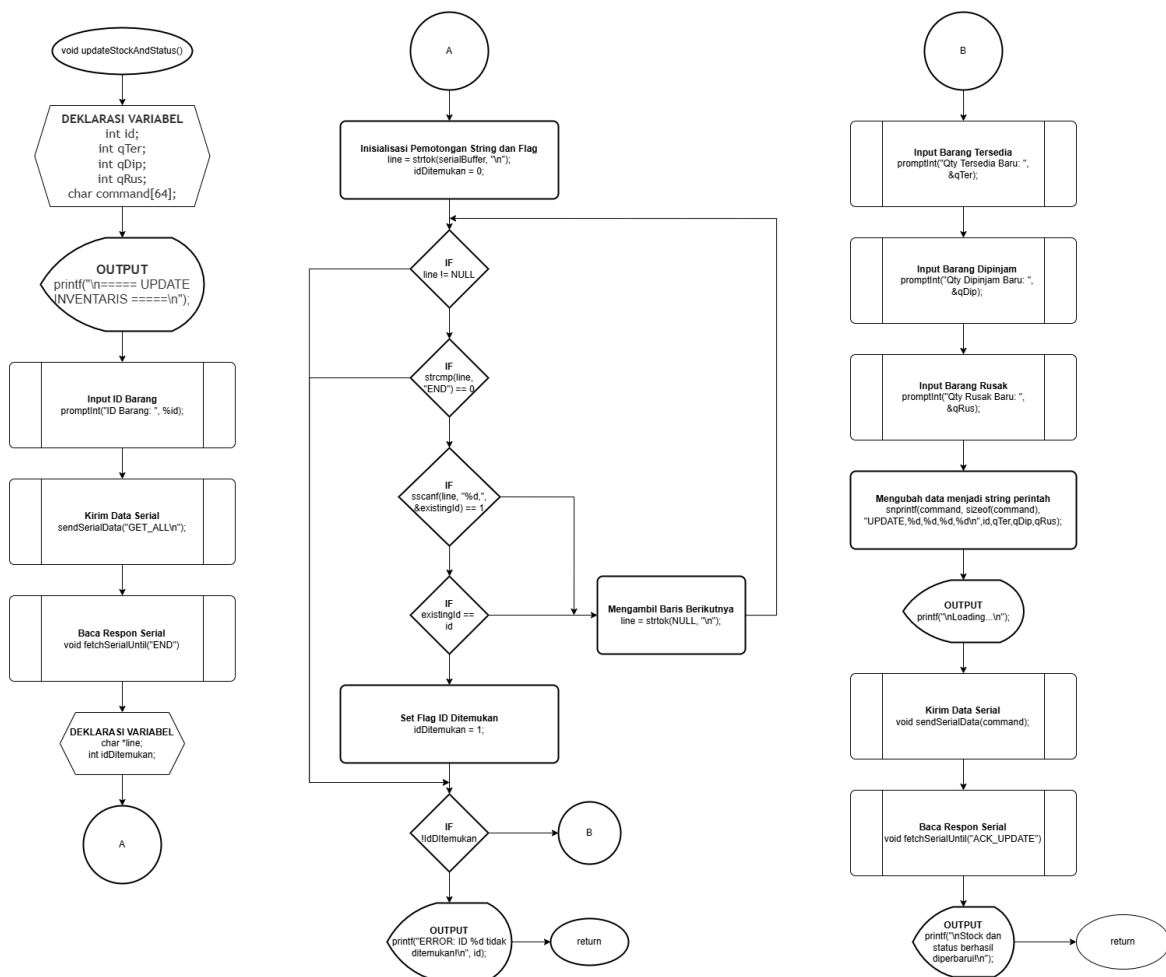
Gambar 3.1.2 Flowchart Fungsi *addInventory*.



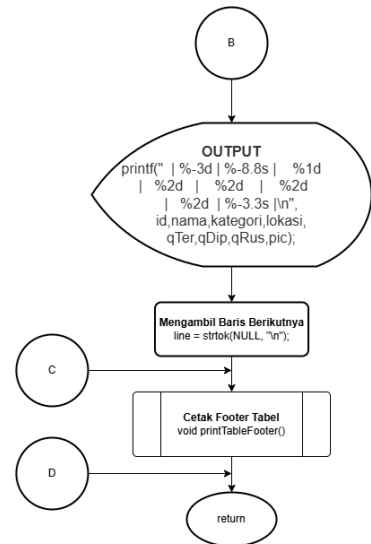
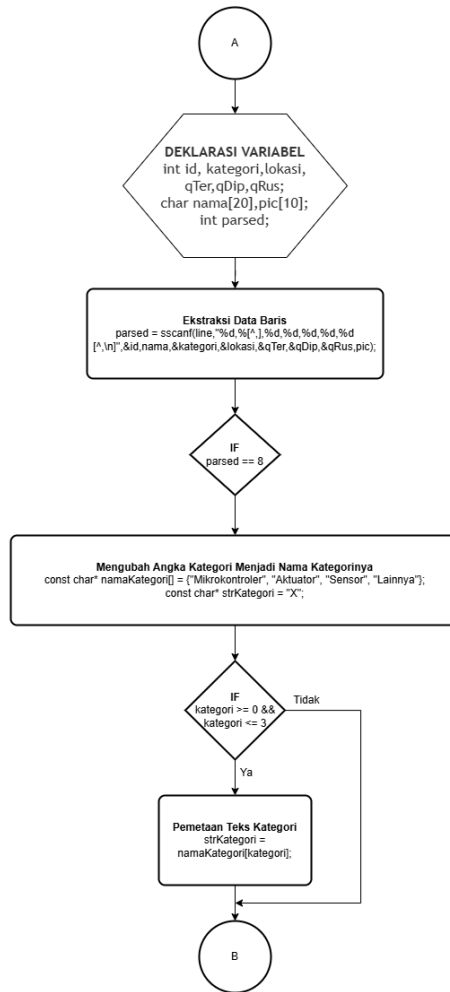
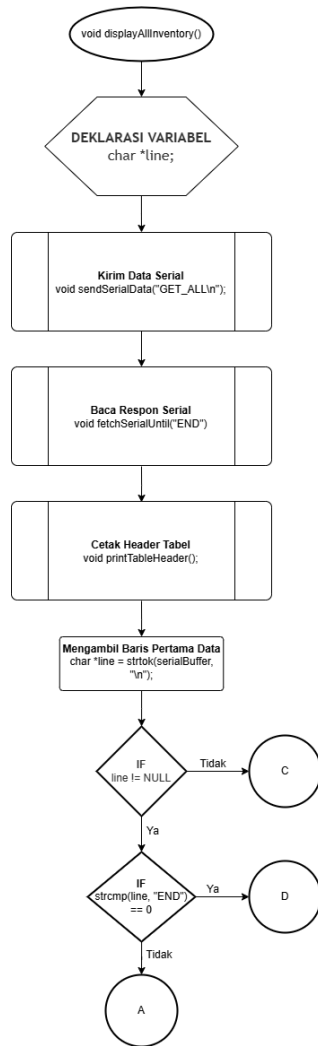
Gambar 3.1.3 Flowchart Fungsi *deleteInventory*.



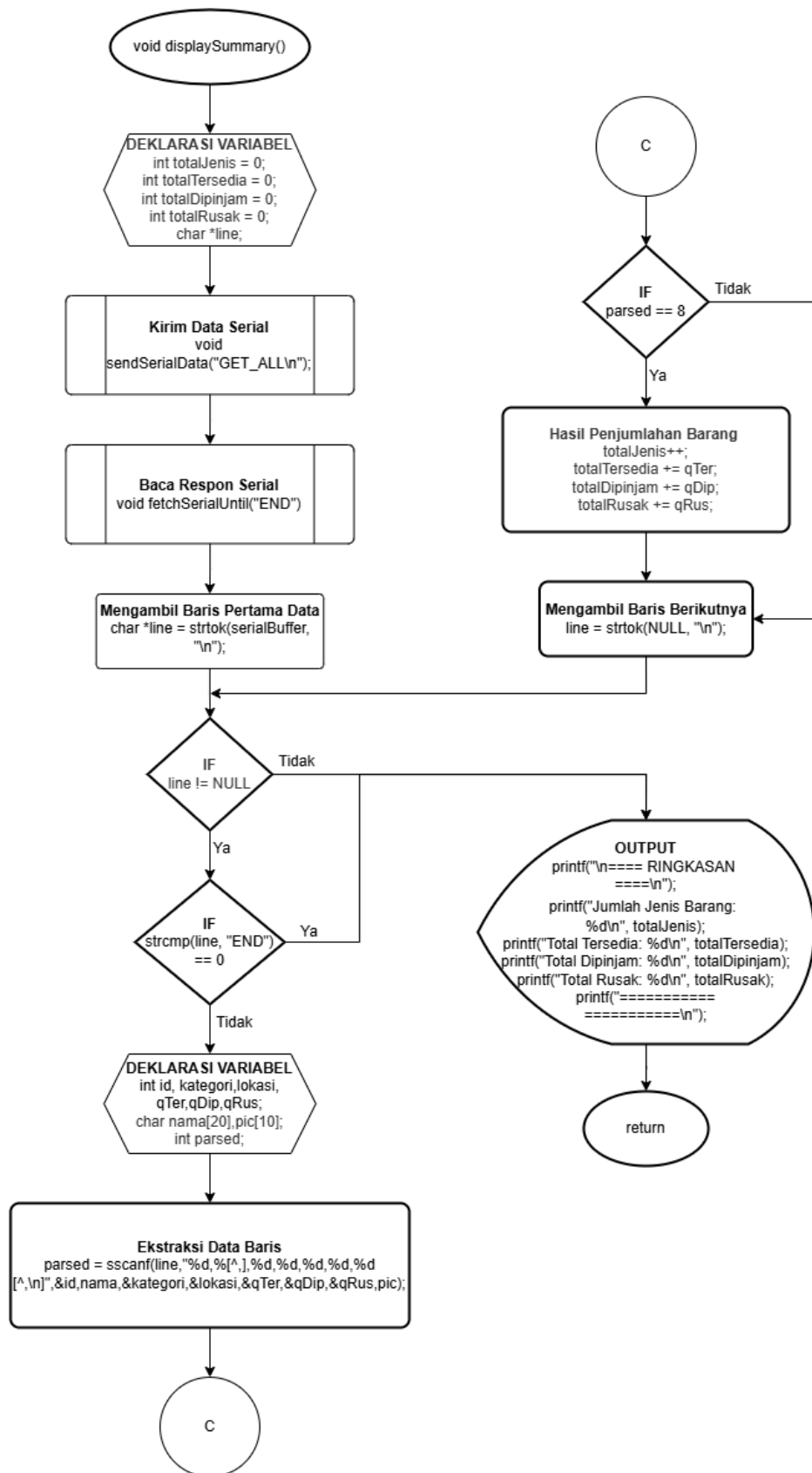
Gambar 3.1.4 Flowchart Fungsi *searchByID*.



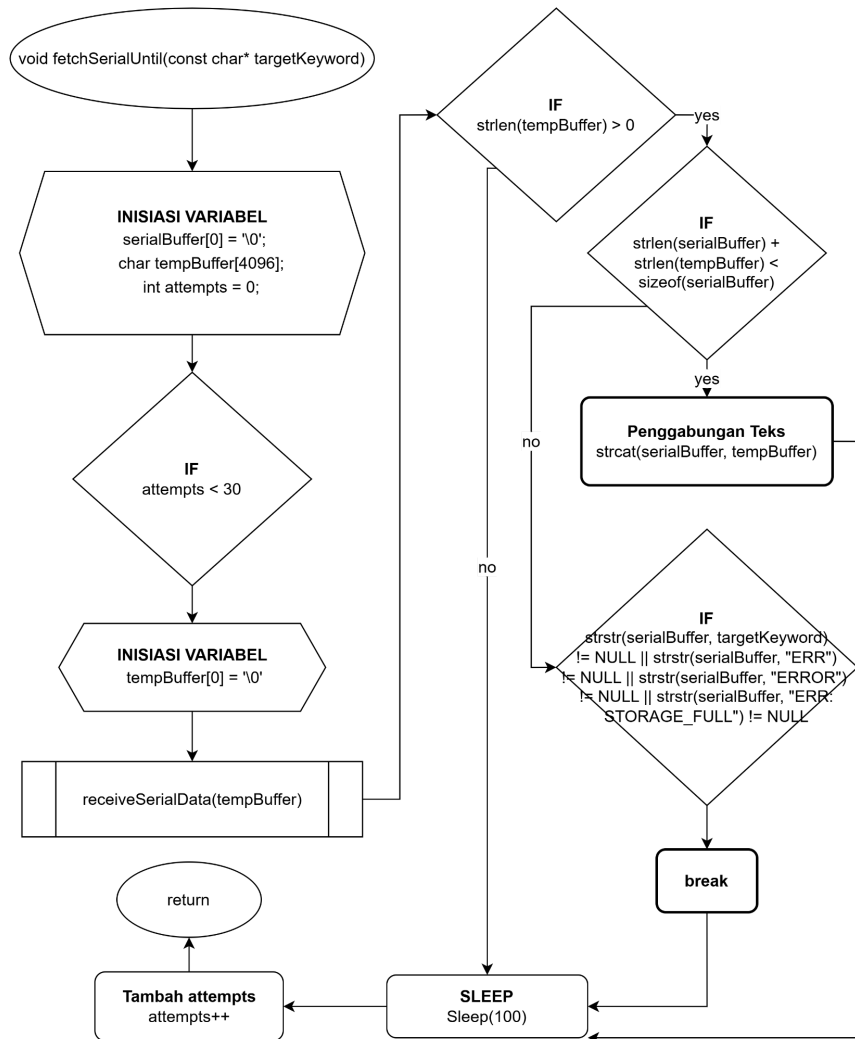
Gambar 3.1.5 Flowchart Fungsi *updateStockAndStatus*.



Gambar 3.1.6 Flowchart Fungsi *displayAllInventory*.

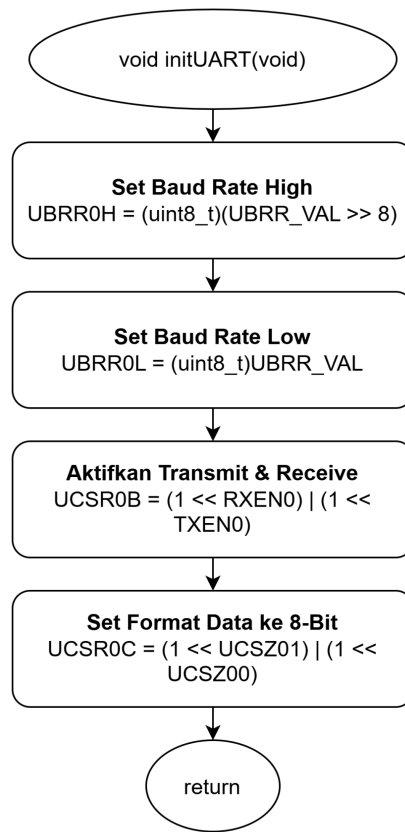


Gambar 3.1.7 Flowchart Fungsi *displaySummary*.

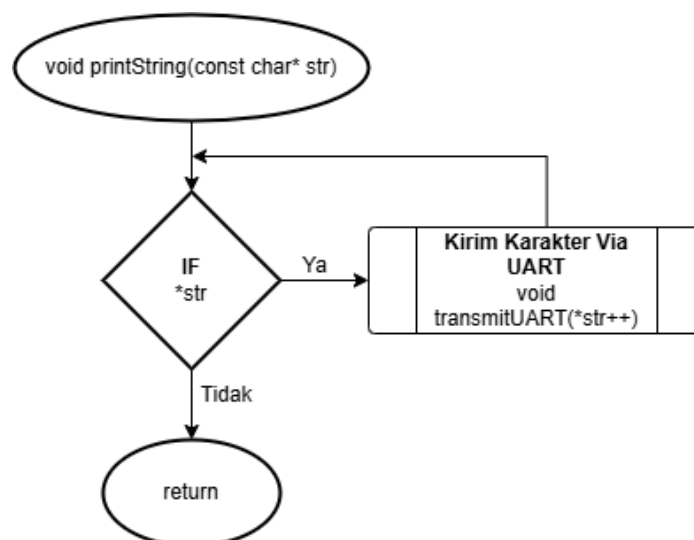


Gambar 3.1.1.8 Flowchart Fungsi *fetchSerialUntil*.

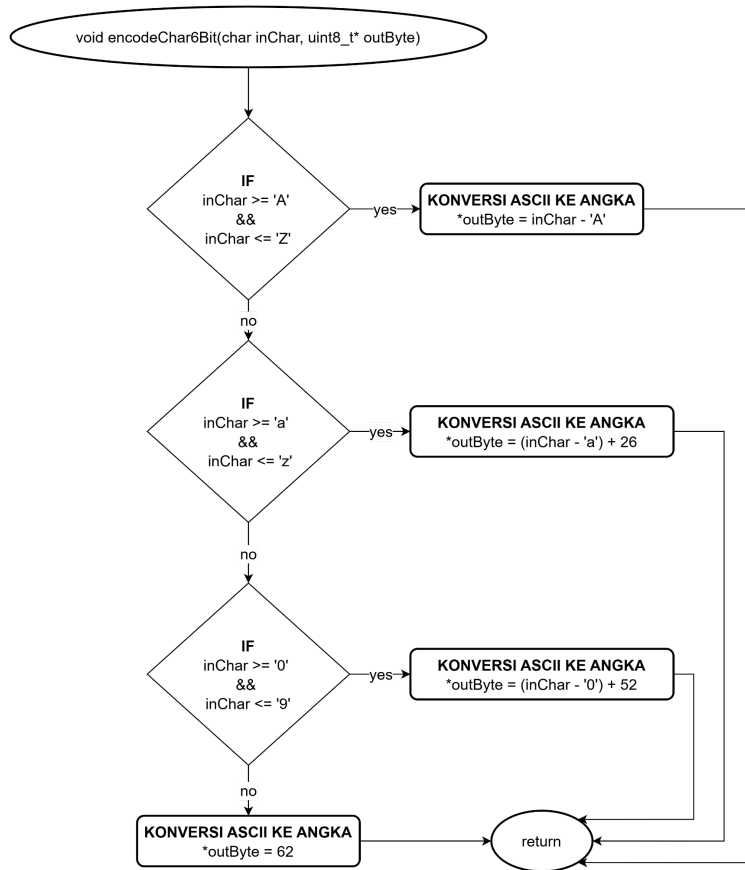
3.1.2 Flowchart database.c



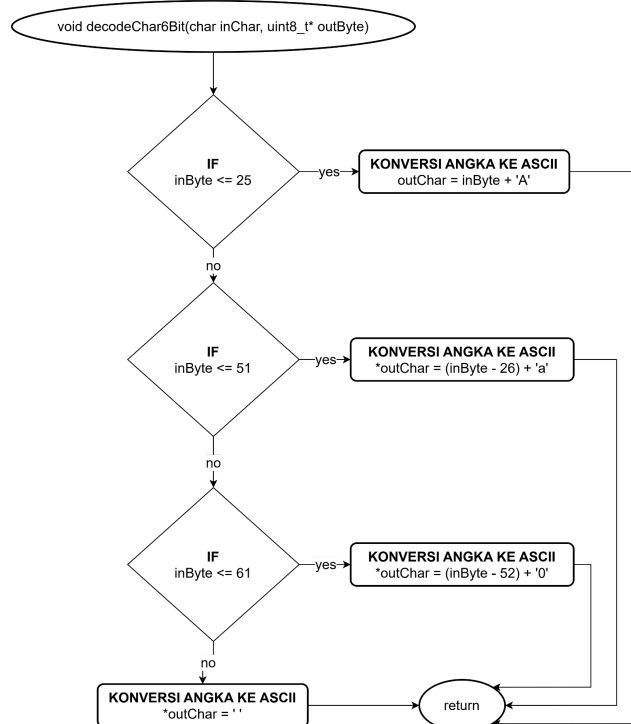
Gambar 3.1.2.1 Flowchart Fungsi *initUART*.



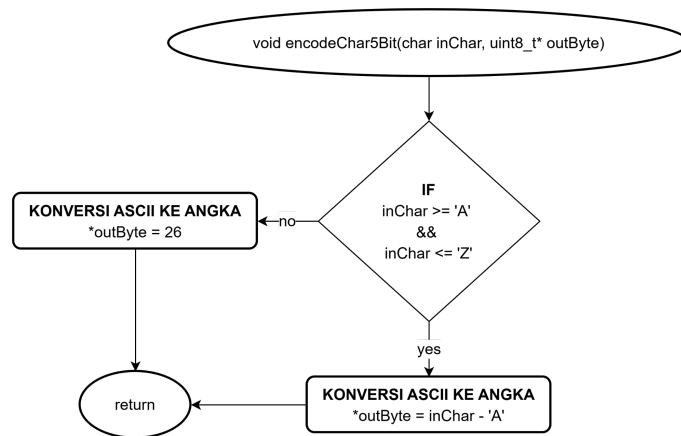
Gambar 3.1.2.2 Flowchart *printString*.



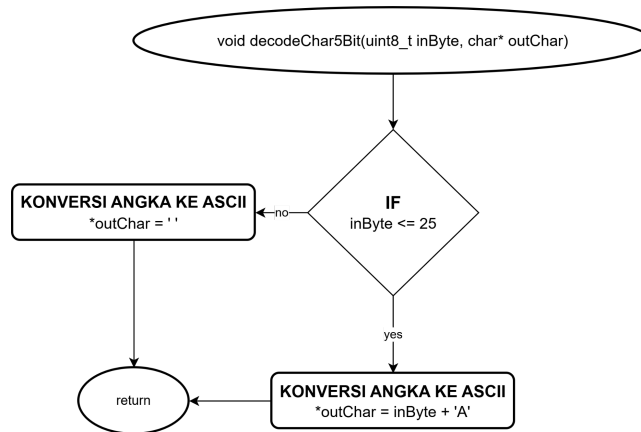
Gambar 3.1.2.5 Flowchart Fungsi *encodeChar6Bit*.



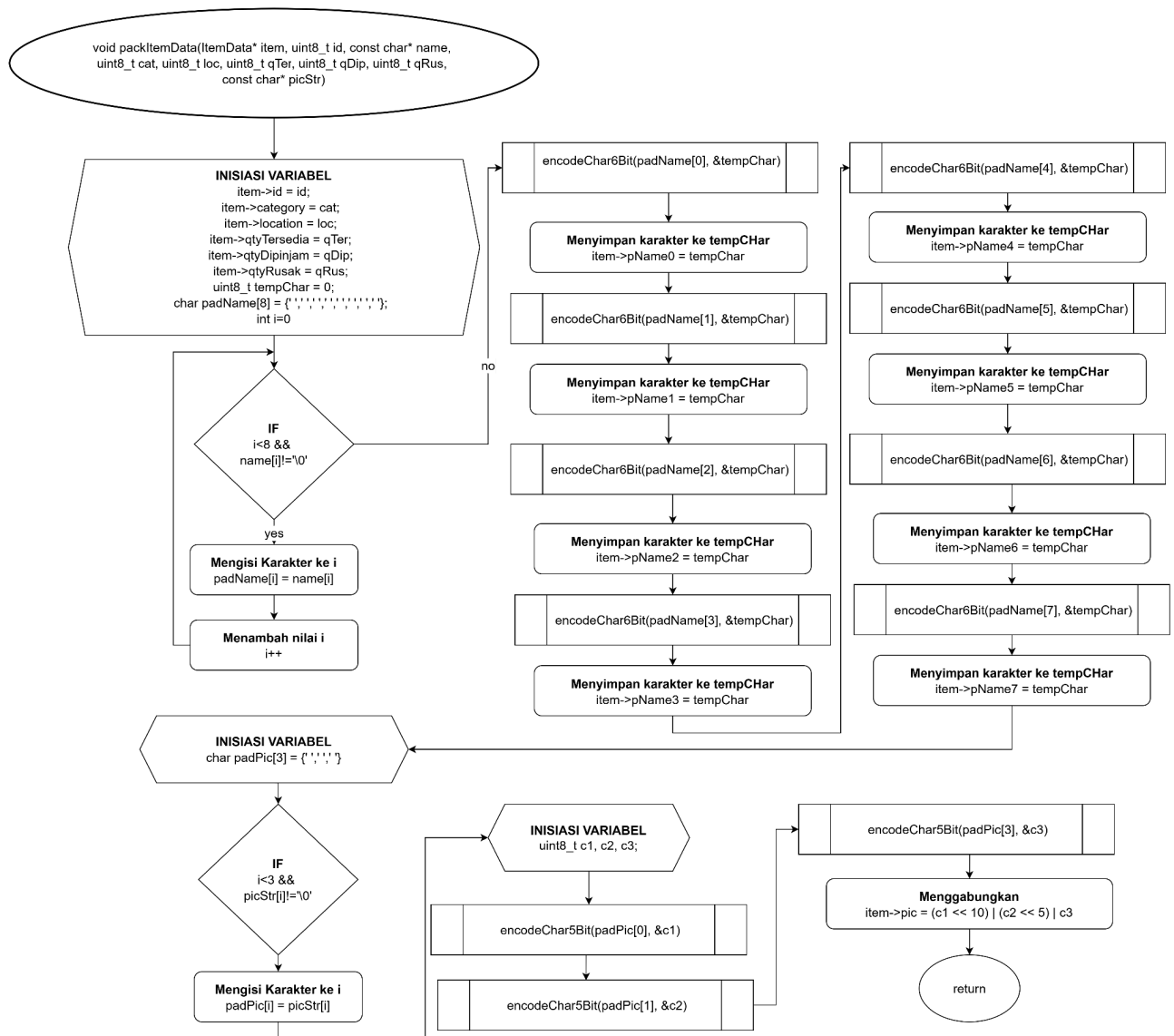
Gambar 3.1.2.6 Flowchart Fungsi *decodeChar6Bit*.



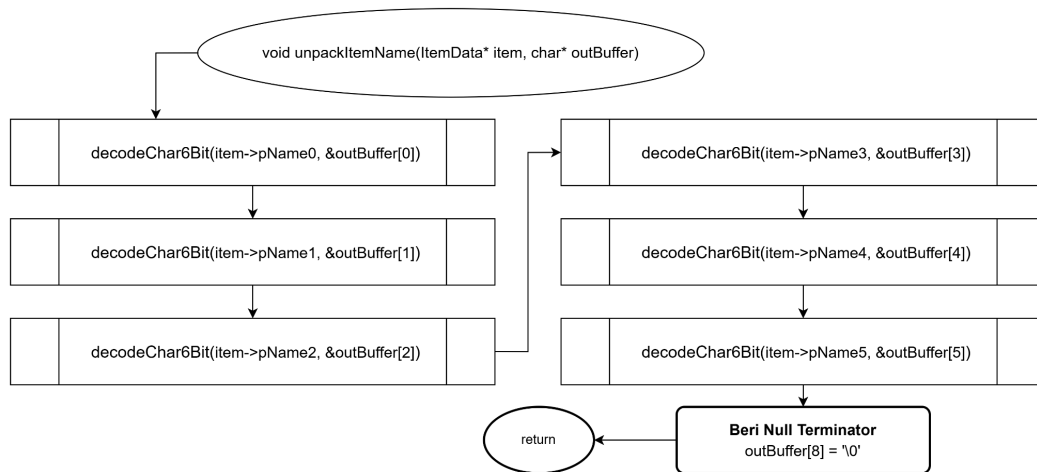
Gambar 3.1.2.7 Flowchart Fungsi *encodeChar5Bit*.



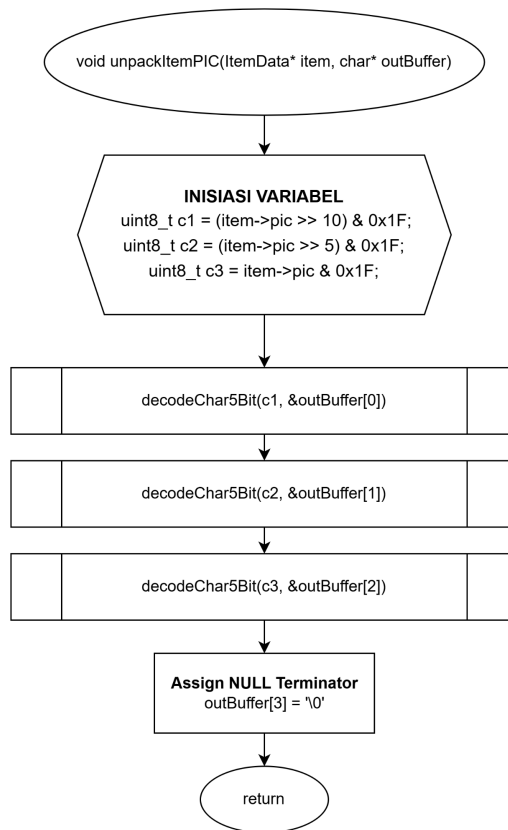
Gambar 3.1.2.8 Flowchart Fungsi *decodeChar5Bit*.



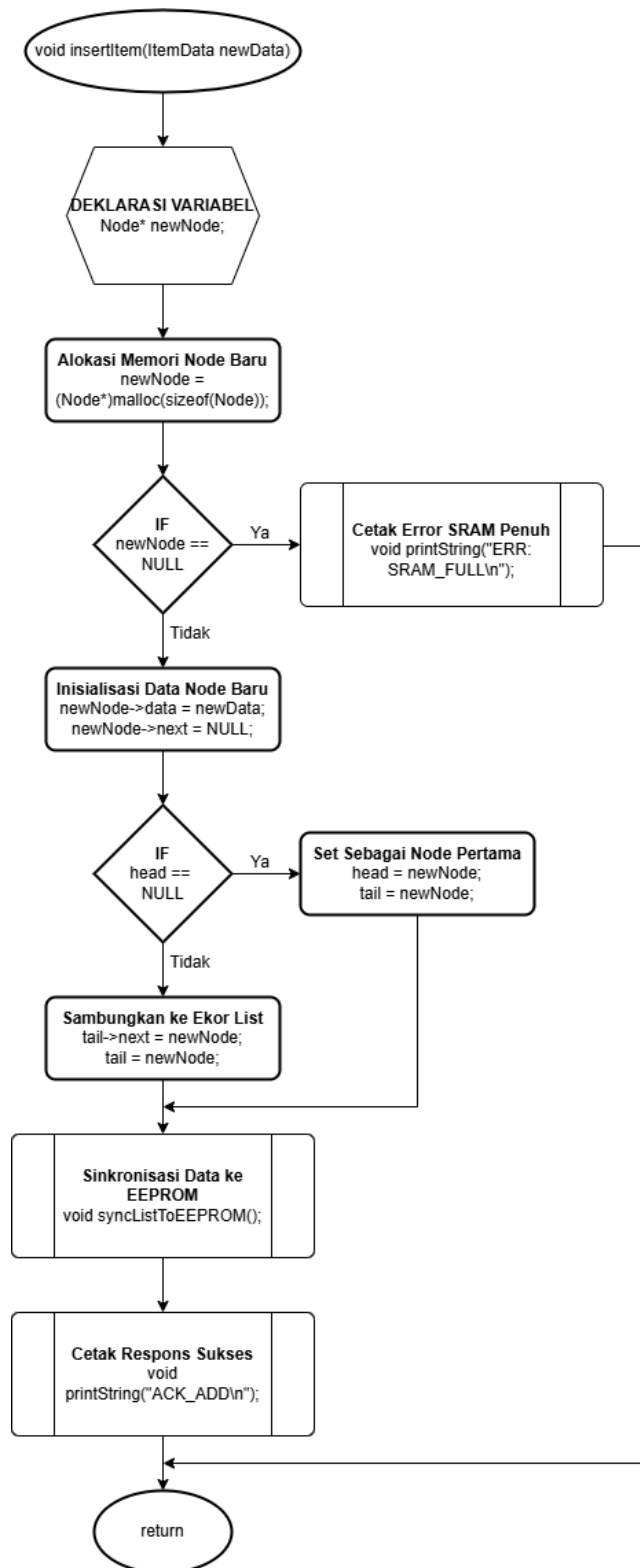
Gambar 3.1.2.9 Flowchart Fungsi *packItemData*.



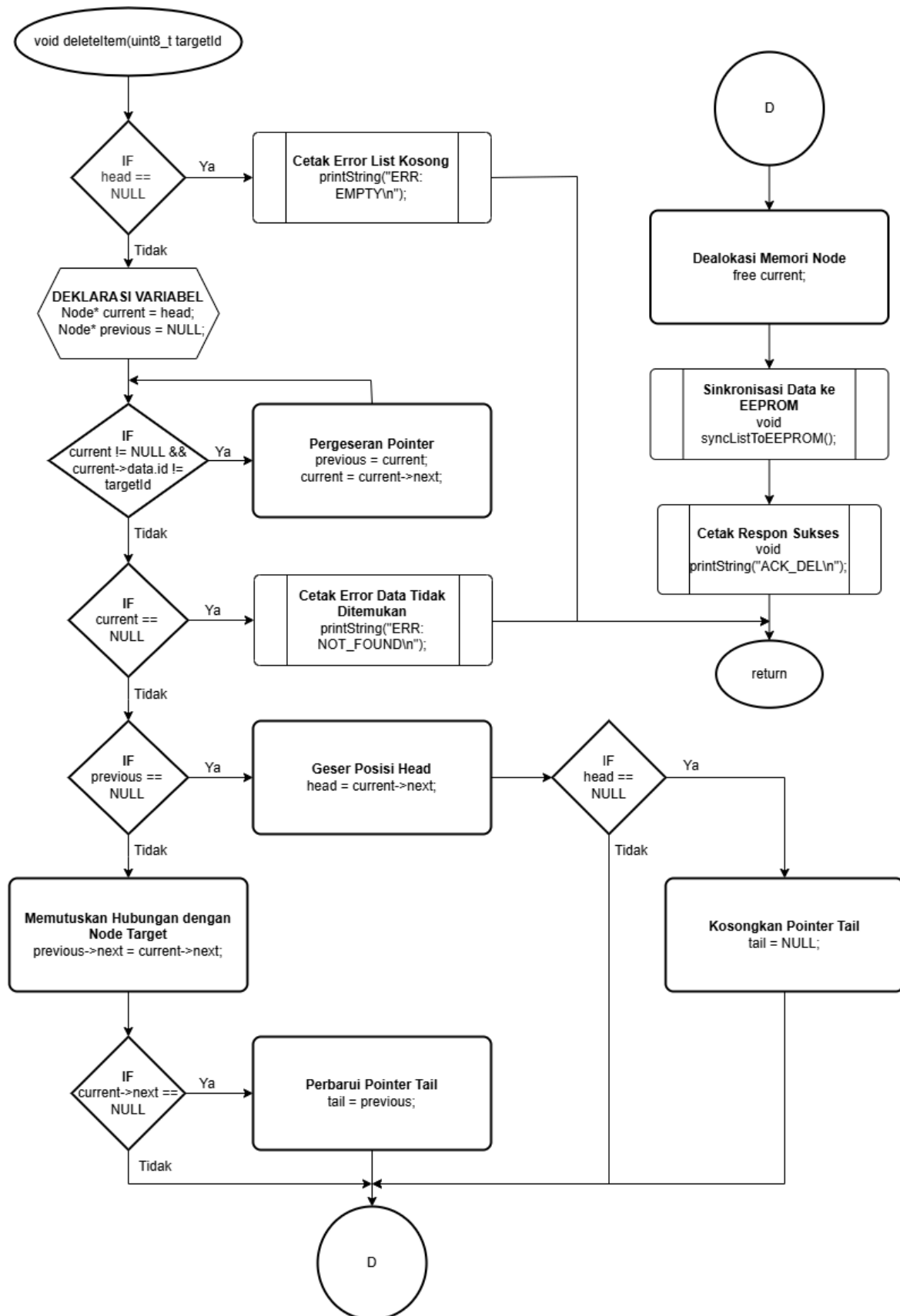
Gambar 3.1.2.10 Flowchart Fungsi *unpackItemName*.



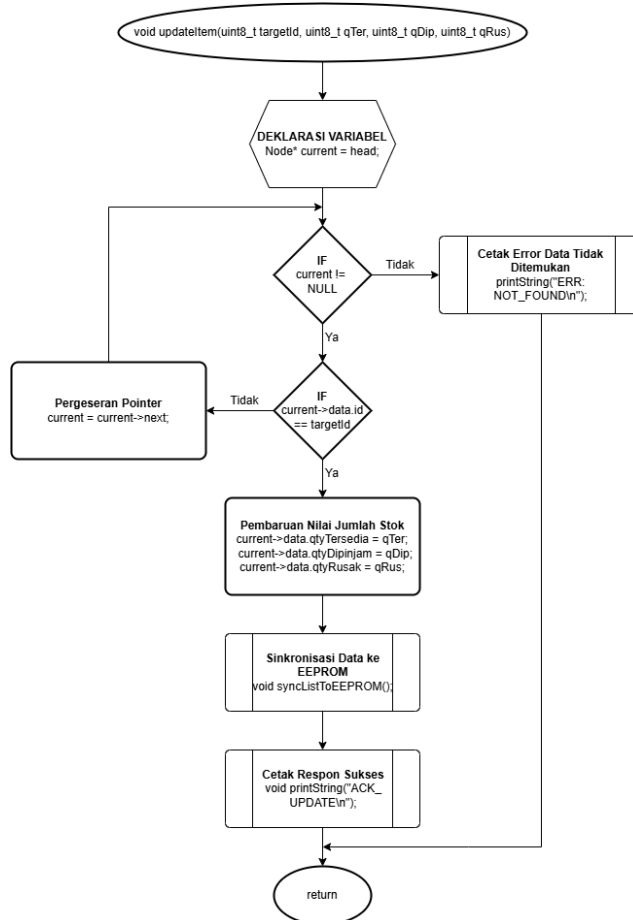
Gambar 3.1.2.11 Flowchart Fungsi *unpackItemPIC*.



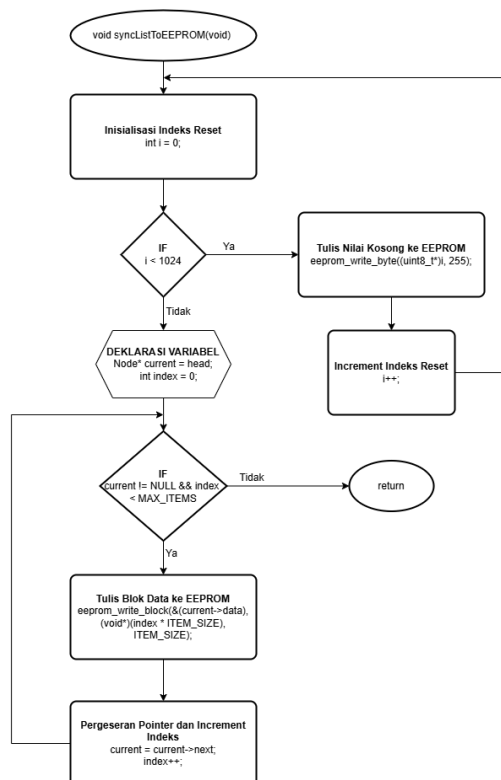
Gambar 3.1.2.12 Flowchart Fungsi *insertItem*.



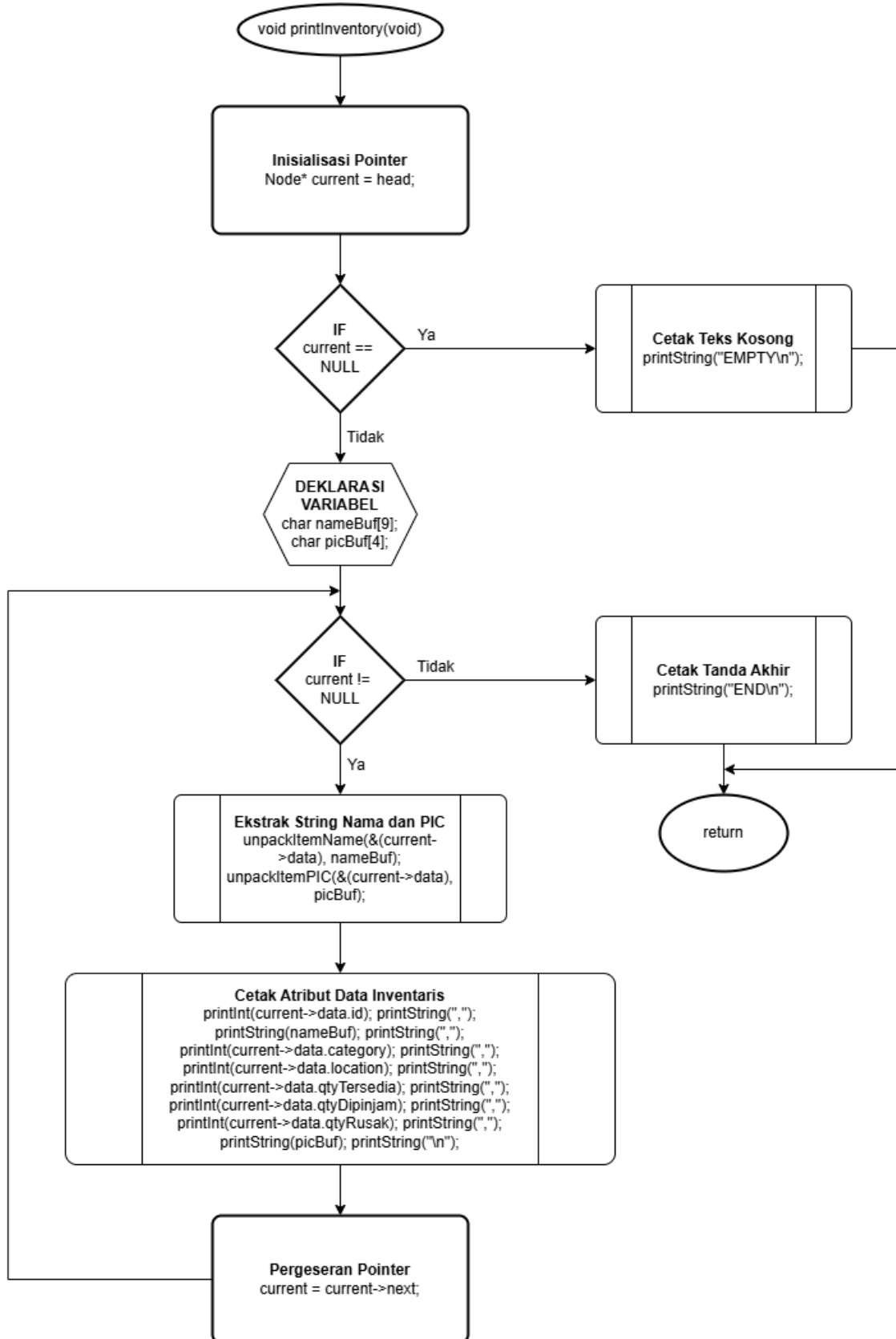
Gambar 3.1.2.13 Flowchart Fungsi *deleteItem*.



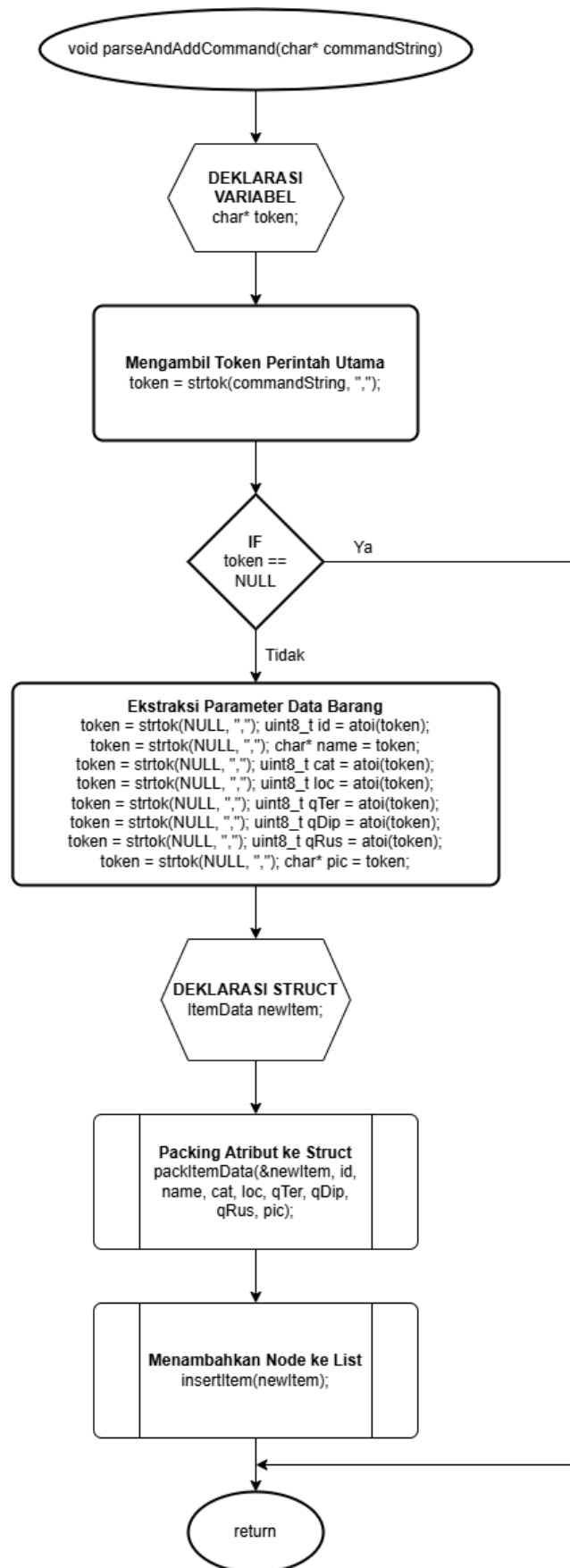
Gambar 3.1.2.14 Flowchart Fungsi *updateItem*.



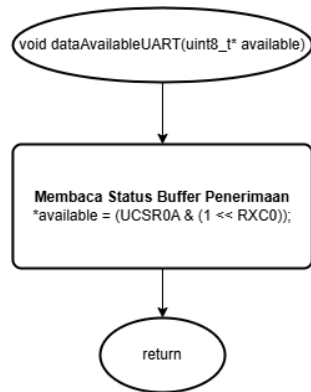
Gambar 3.1.2.15 Flowchart Fungsi *syncListToEEPROM*.



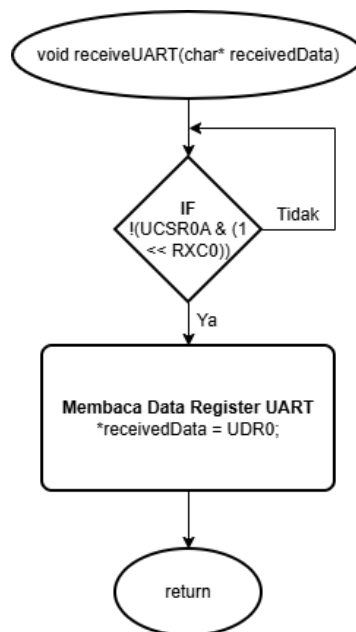
Gambar 3.1.2.16 Flowchart Fungsi *printInventory*.



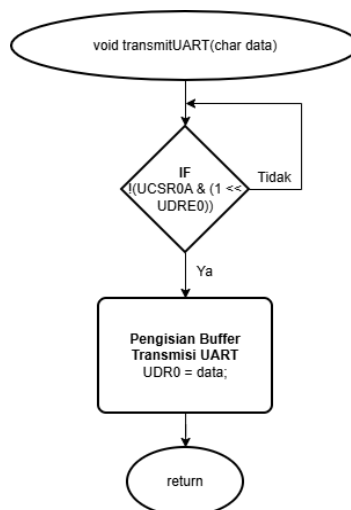
Gambar 3.1.2.17 Flowchart Fungsi *parseAndAddCommand*.



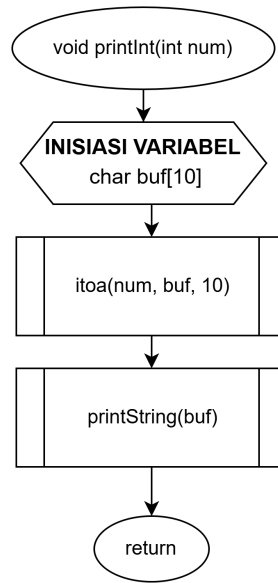
Gambar 3.1.2.18 Flowchart Fungsi *dataAvailableUART*.



Gambar 3.1.2.19 Flowchart Fungsi *receiveUART*.

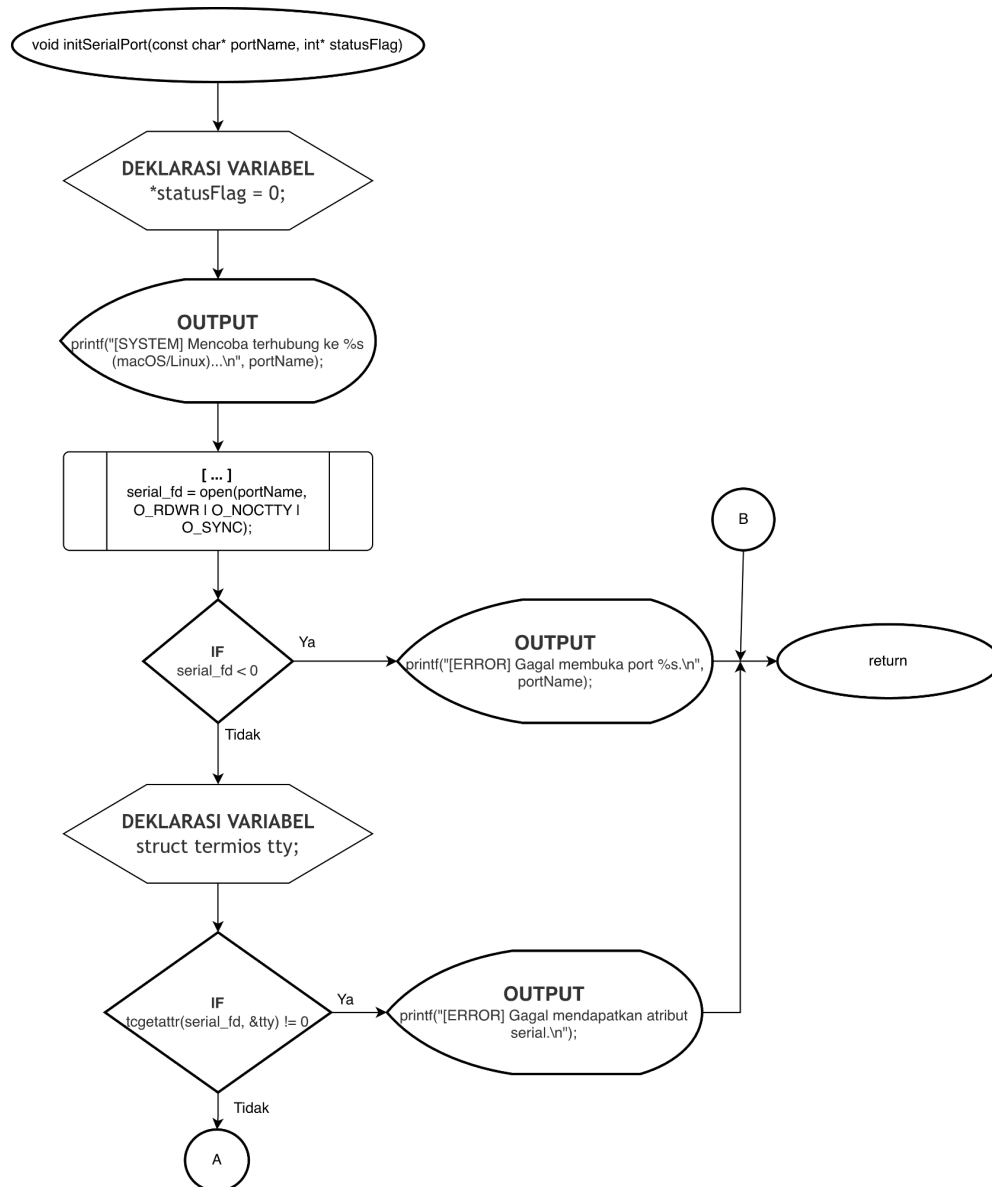


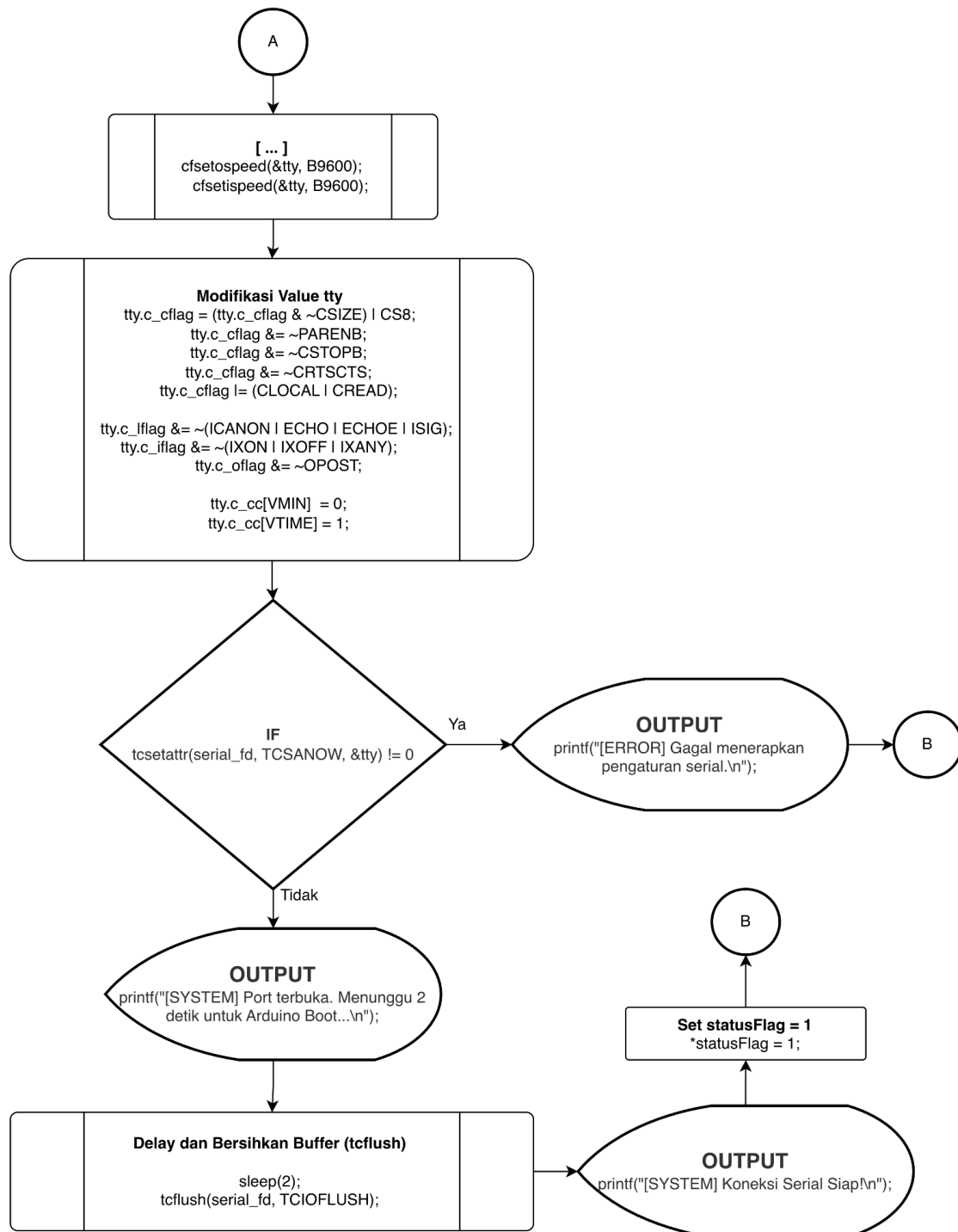
Gambar 3.1.2.20 Flowchart Fungsi *transmitUART*.



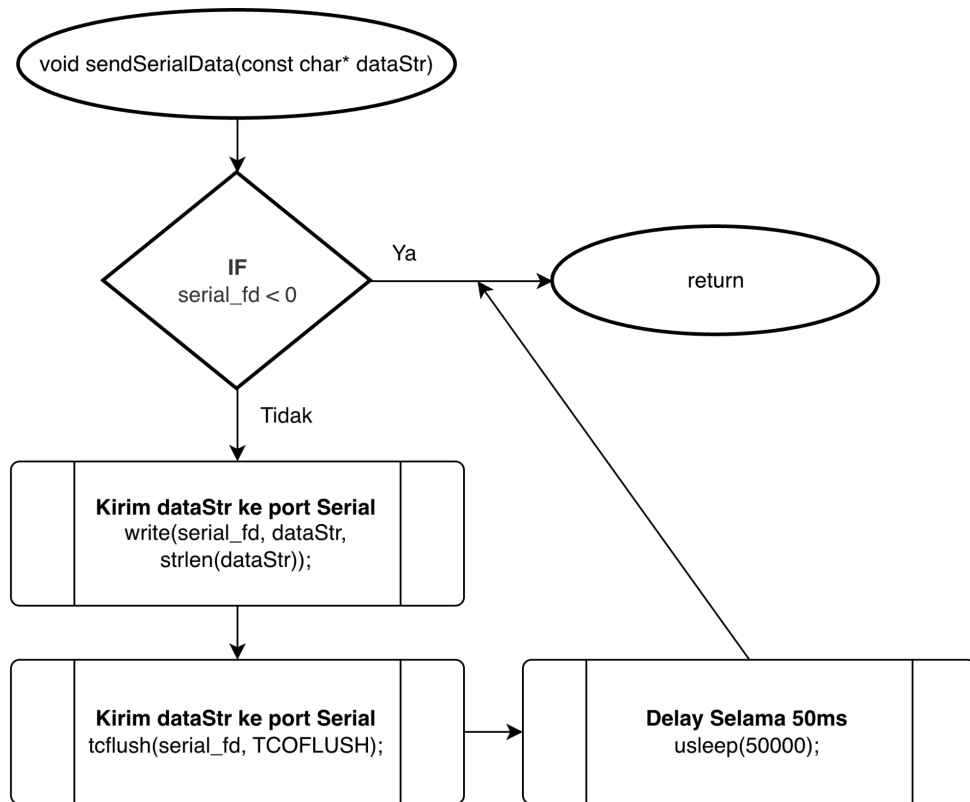
Gambar 3.1.2.21 Flowchart Fungsi *printInt*.

3.1.3 Flowchart serial_comm.c

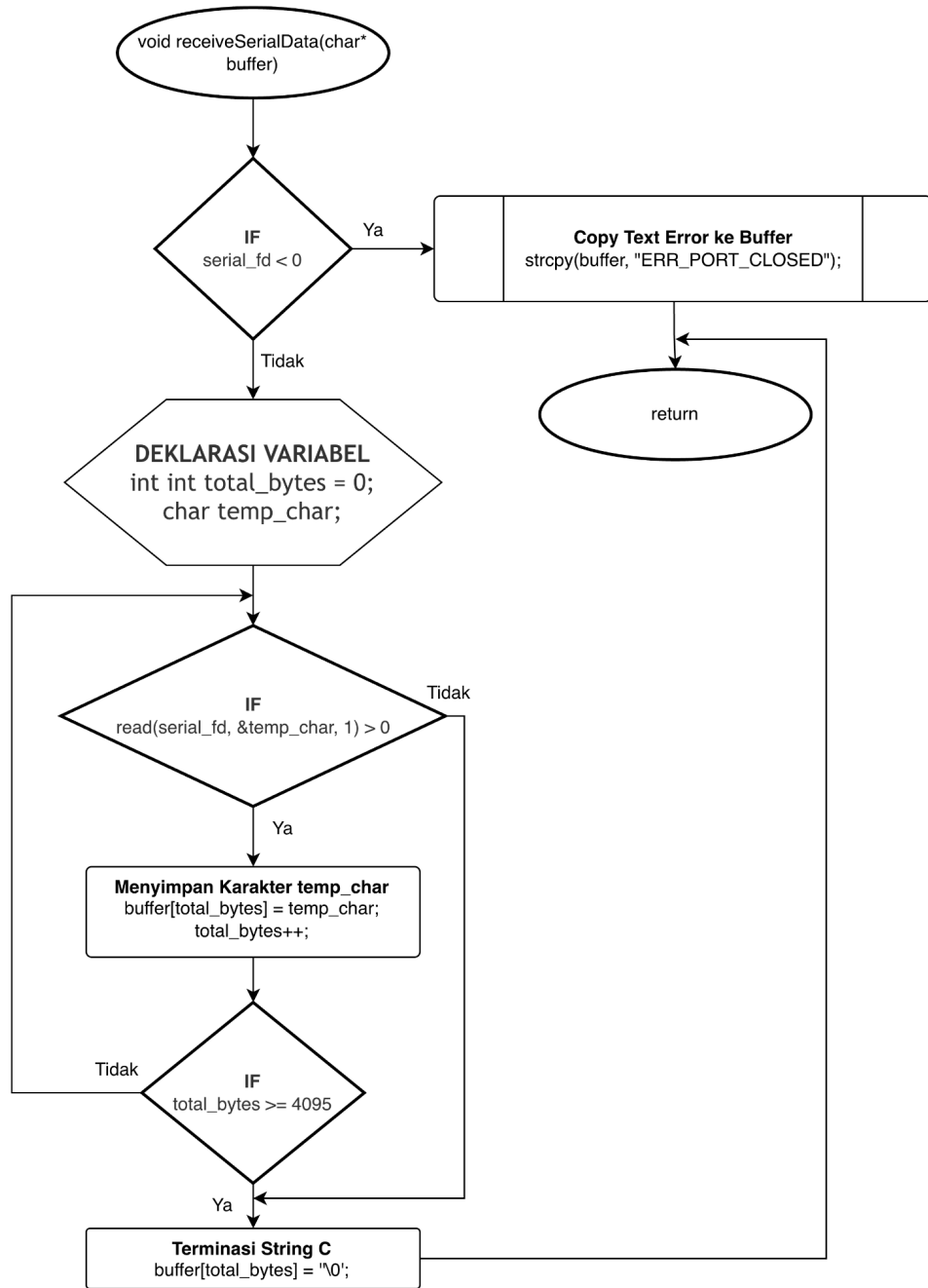




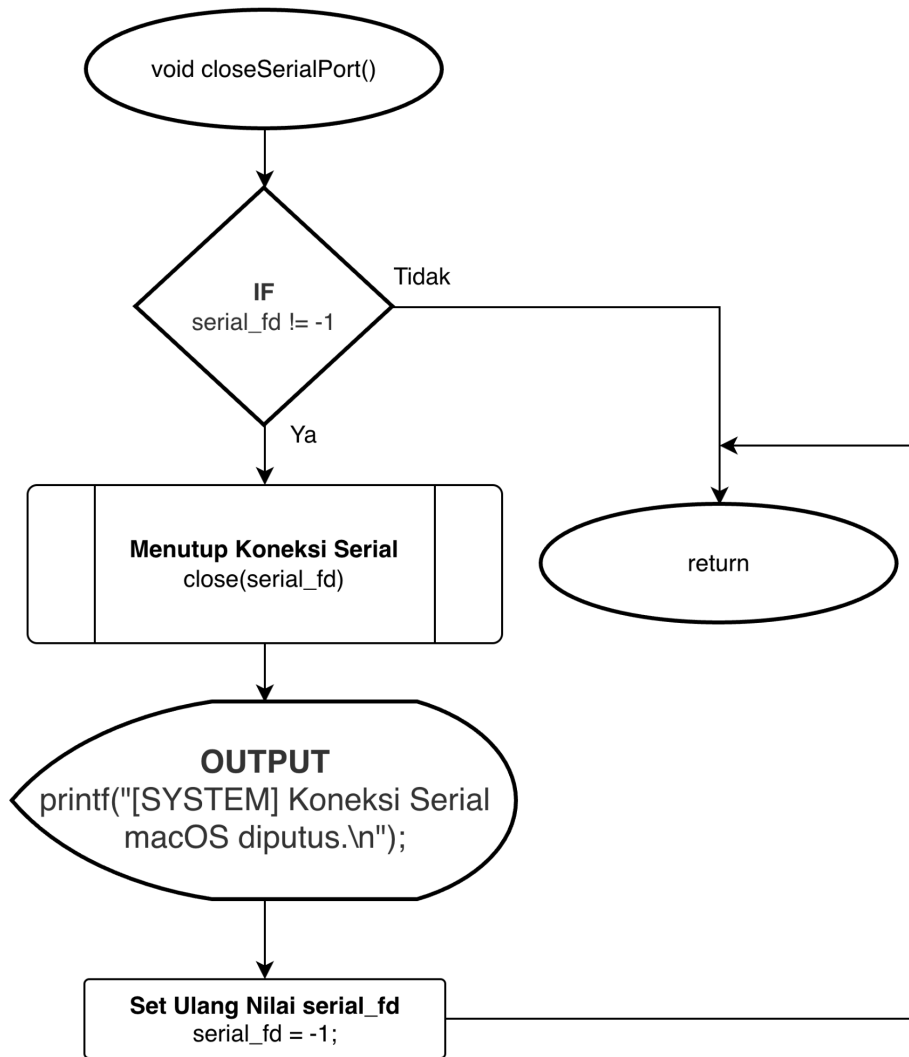
Gambar 3.1.3.1. Flowchart Fungsi *initSerialPort*.



Gambar 3.1.3.2. Flowchart Fungsi *sendSerialData*.

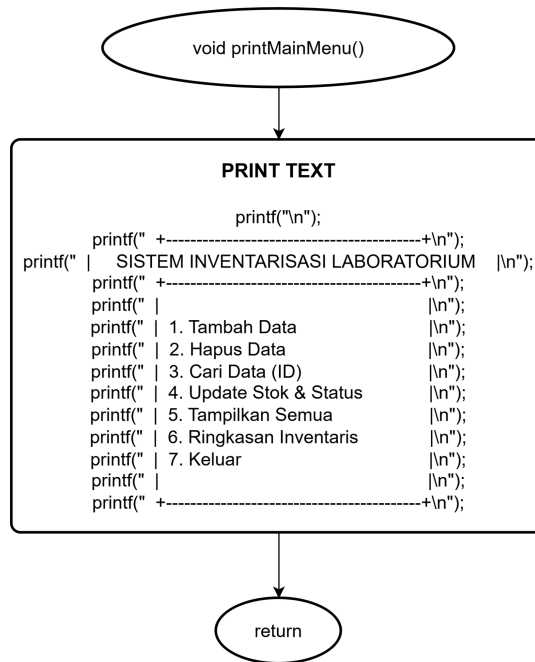


Gambar 3.1.3.3. Flowchart Fungsi *receiveSerialData*.

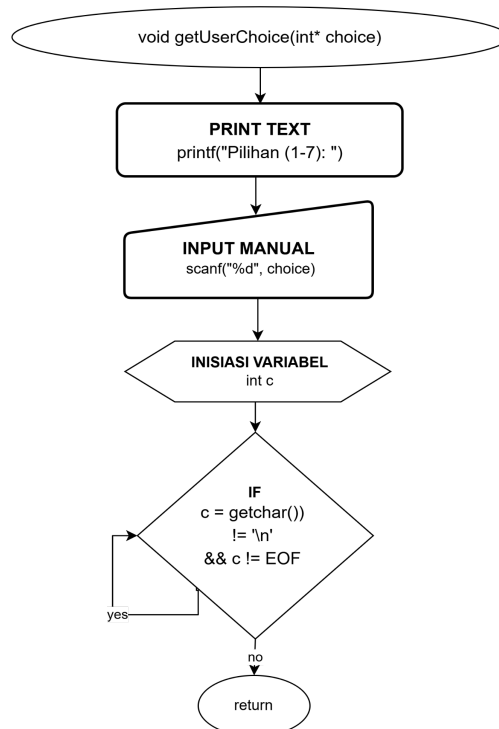


Gambar 3.1.3.4. Flowchart Fungsi *closeSerialPort*.

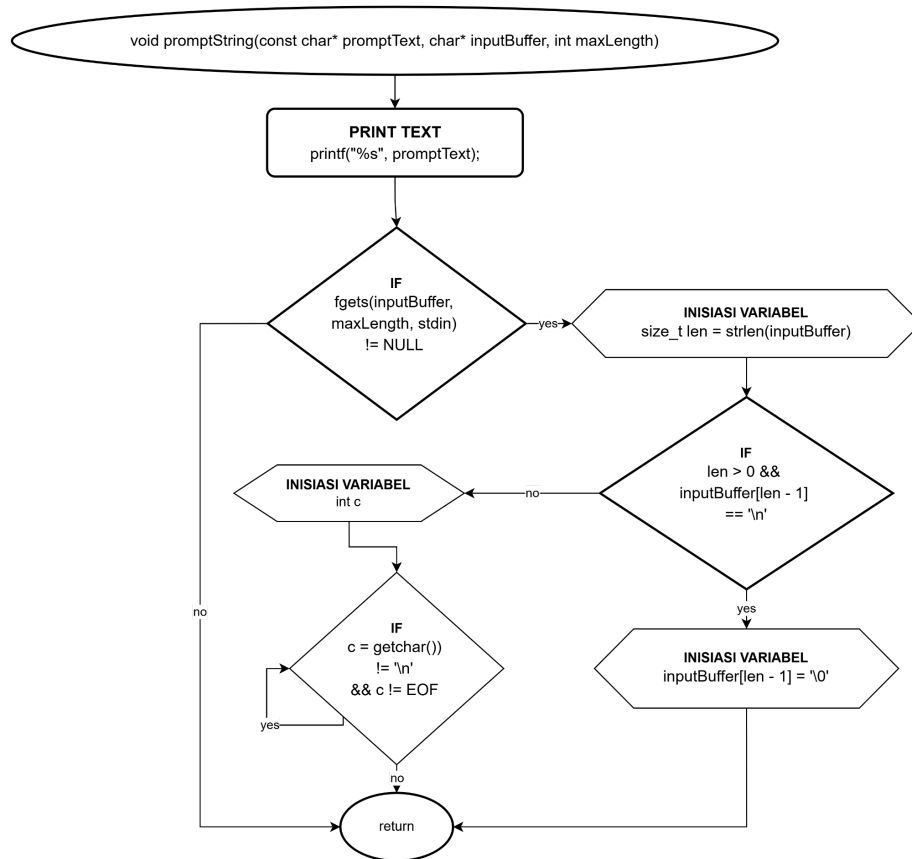
3.1.4 Flowchart user_interface.c



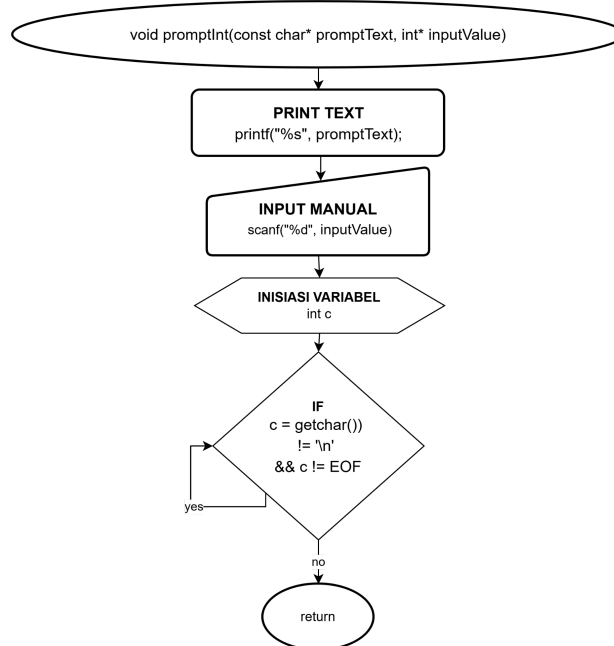
Gambar 3.1.4.1 Flowchart Fungsi *printMainMenu*.



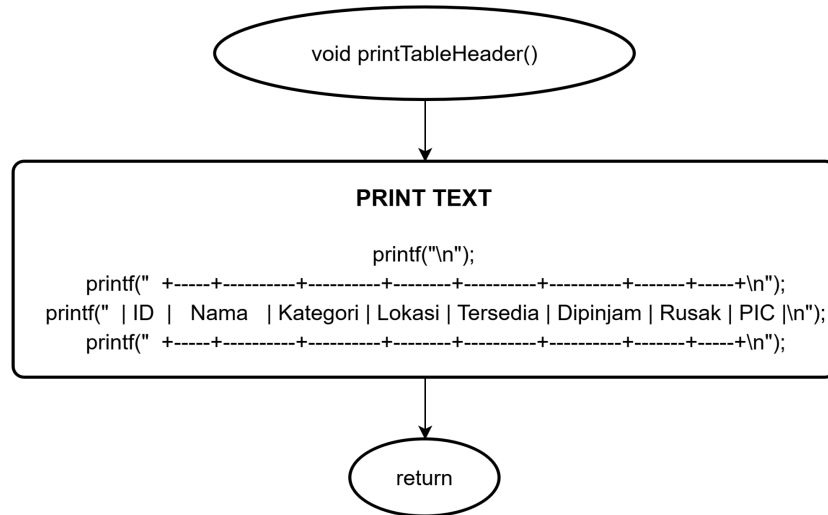
Gambar 3.1.4.2 Flowchart Fungsi *getUserChoice*.



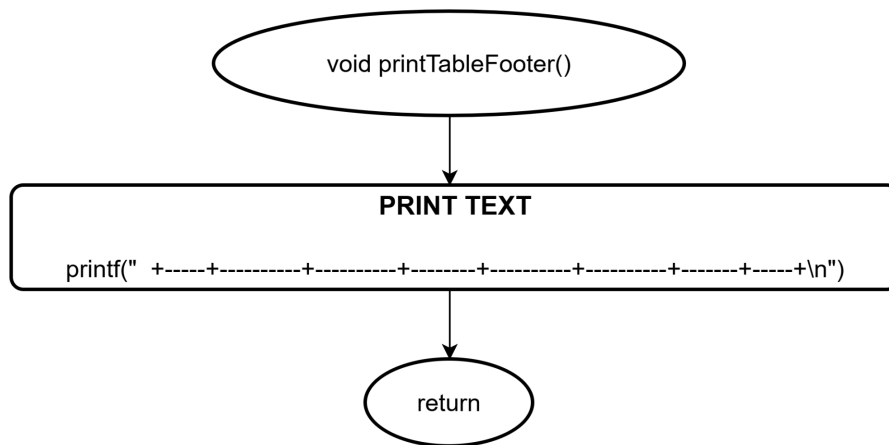
Gambar 3.1.4.3 Flowchart Fungsi *promptString*.



Gambar 3.1.4.4 Flowchart Fungsi *promptInt*.



Gambar 3.1.4.5 Flowchart Fungsi *printTableHeader*.



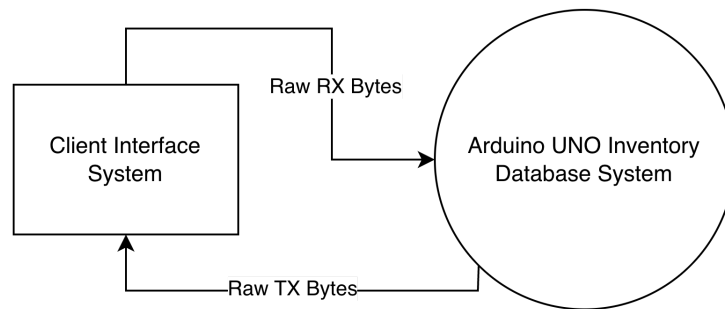
Gambar 3.1.4.6 Flowchart Fungsi *printTableFooter*.

3.2 Data Flow Diagram (DFD)

Akan dibuat 2 DFD berbeda, yaitu DFD dari *Arduino UNO System* dan DFD dari *Client Interface System*. Berikut adalah rinciannya:

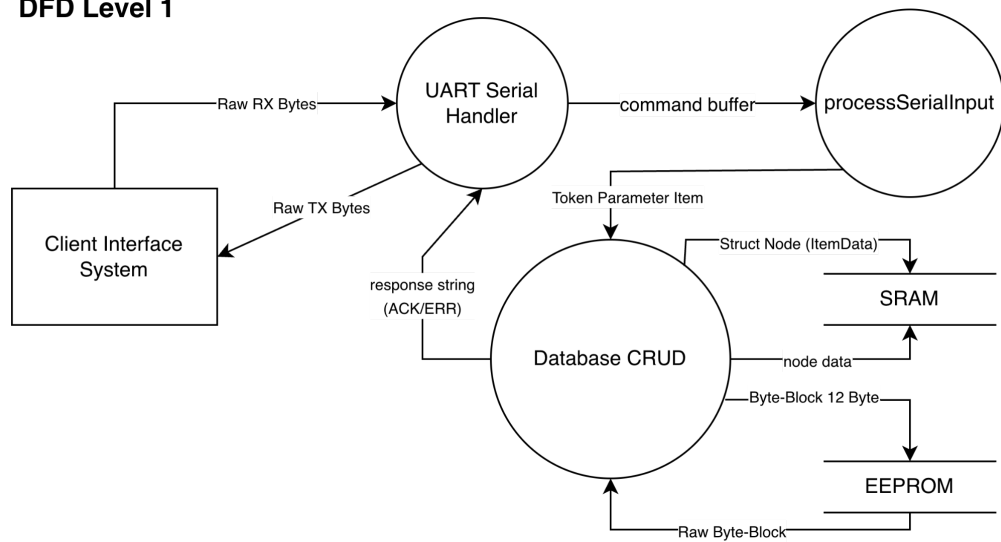
3.2.1 DFD Arduino UNO System

DFD Level 0



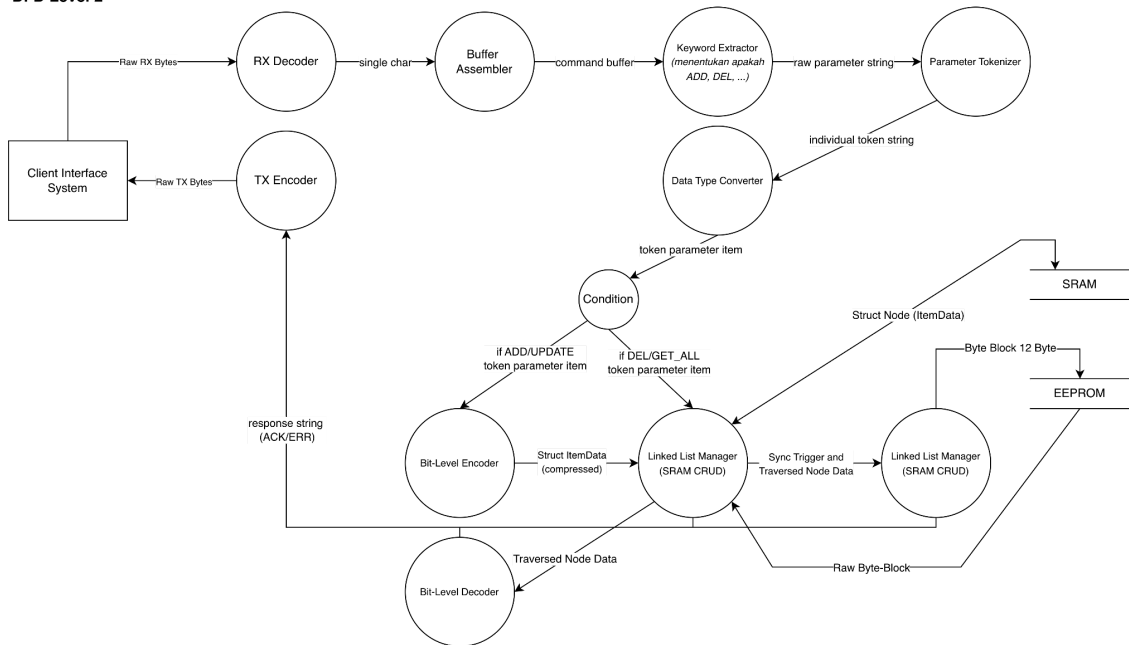
Gambar 3.2.1.1. DFD Level 0 *Arduino UNO System*.

DFD Level 1



Gambar 3.2.1.2. DFD Level 1 *Arduino UNO System*.

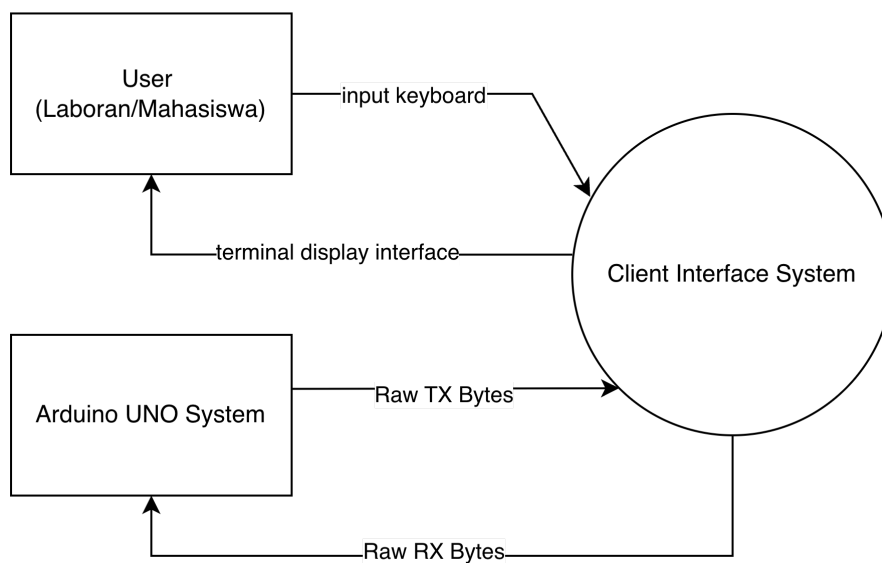
DFD Level 2



Gambar 3.2.1.3. DFD Level 2 *Arduino UNO System*.

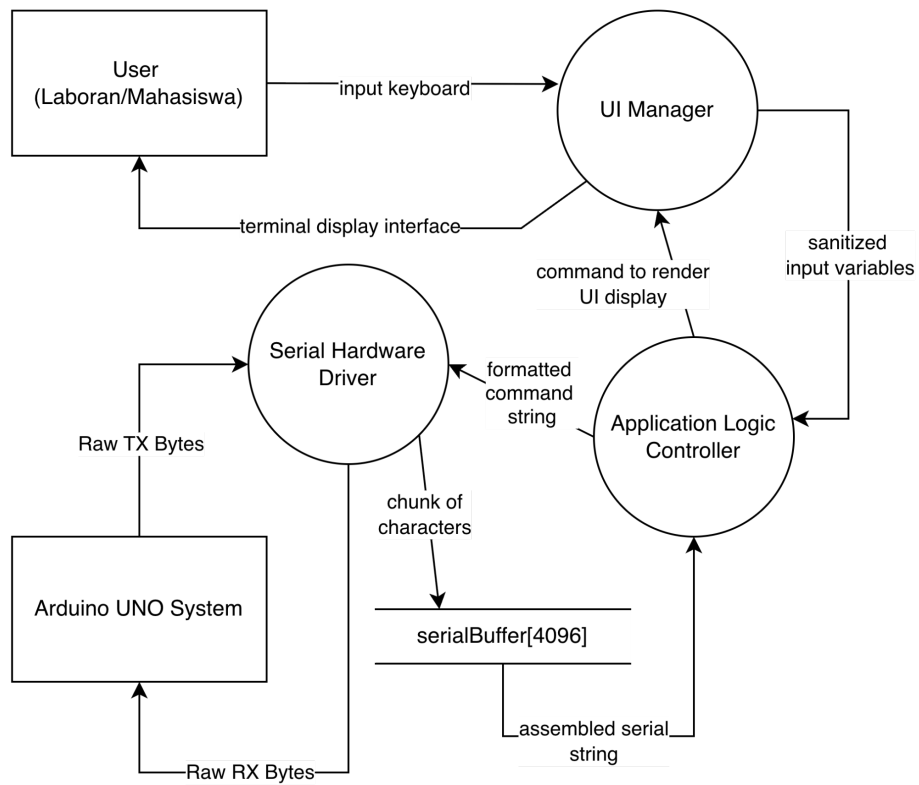
3.2.2 DFD Client Interface System

DFD Level 0

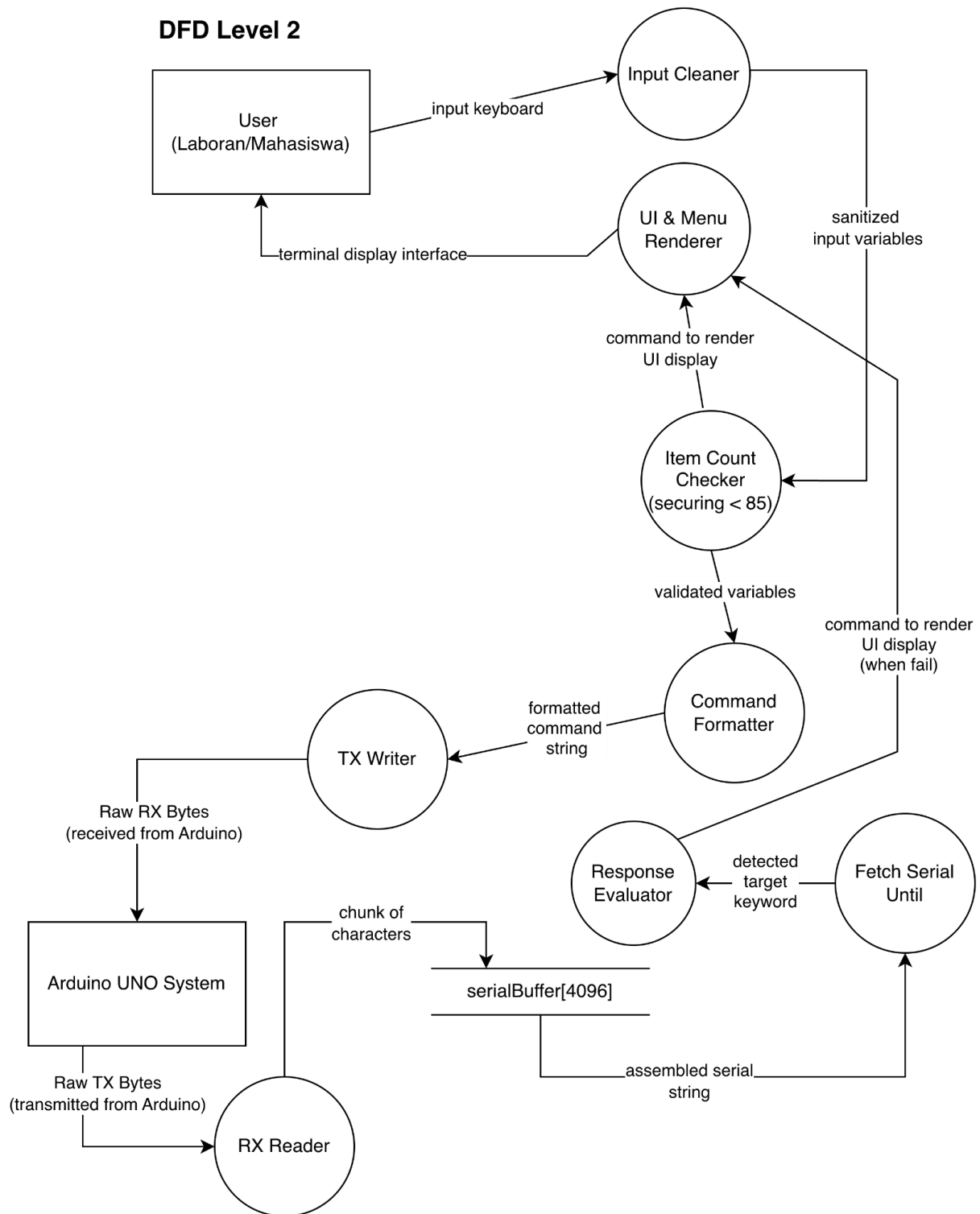


Gambar 3.2.2.1. DFD Level 0 *Client Interface System*.

DFD Level 1



Gambar 3.2.2.2. DFD Level 1 *Client Interface System*.



Gambar 3.2.2.3. DFD Level 2 *Client Interface System*.

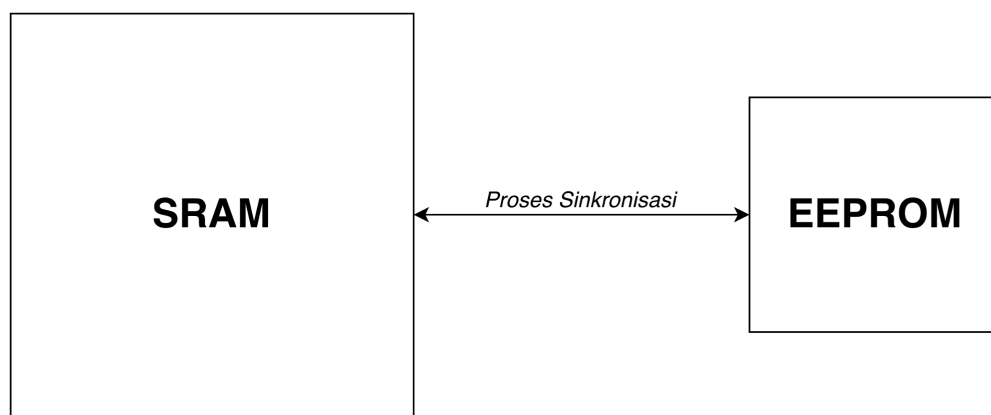
IV. Penjelasan Logika dan Penjelasan Penggunaan Struktur Data

4.1 Struktur Penggunaan Memori Pada Arduino

Arsitektur model penyimpanan memori pada program ini dirancang untuk memaksimalkan keterbatasan perangkat keras mikrokontroler ATmega328P pada

Arduino Uno, yang secara aktual hanya memiliki dua jenis memori utama, yaitu SRAM sebesar 2 KB dan EEPROM sebesar 1 KB. Untuk mencapai kinerja optimal tanpa mengorbankan integritas data itu sendiri, sistem kami menerapkan pemisahan tugas dari masing-masing bagian memori.

SRAM dialokasikan sebagai memori *runtime* (memori aktif yang bersifat *volatile*) untuk menangani operasi algoritma pencarian, penambahan, dan penghapusan data secara cepat. EEPROM sendiri bertindak sebagai memori *non-volatile* (penyimpanan permanen) yang menjamin data inventaris laboratorium embedded systems tidak hilang saat perangkat kehilangan daya (tombol merah di klik ataupun dicabut dari *power supply*).

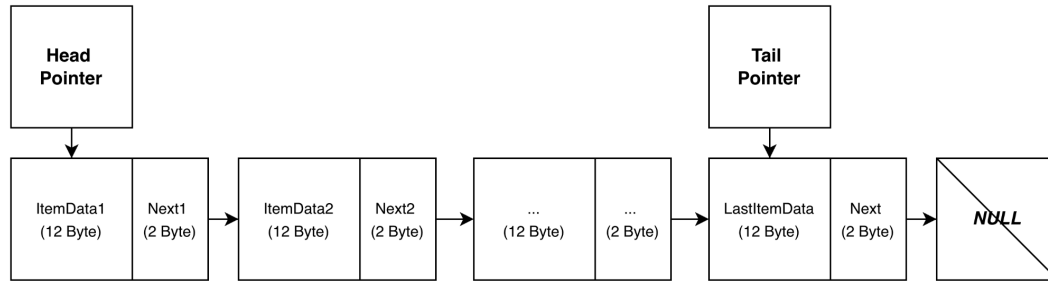


Gambar 4.1. Diagram Blok Umum Antara SRAM dan EEPROM.

4.2 Struktur Data Pada Memori *Volatile*

Saat Arduino beroperasi secara aktif, data dikelola di dalam SRAM menggunakan struktur data *Singly Linked List* dengan konsep *FIFO (First-In, First-Out)*. Pendekatan ini memungkinkan operasi penambahan dan penghapusan data secara dinamis ($O(1)$) untuk penambahan dengan menggunakan *tail pointer*, dan pencarian sekuensial untuk penghapusan suatu node dengan target ID spesifik).

Setiap elemen dalam *Linked List* dibungkus dalam sebuah struktur *Node* yang berisi dua komponen, yaitu data inventaris aktual sebesar 12 *byte* dan sebuah *pointer* memori 16-bit (2 *byte*) yang menunjuk ke *node* berikutnya. Hal ini menghasilkan jejak memori sebesar 14 *byte* per item di dalam SRAM. Pada kapasitas maksimum 85 item, ukuran *Linked List* akan memakan sekitar 1.190 *byte* (~58% kapasitas SRAM), menyisakan ruang yang masih sangat aman untuk alokasi *buffer* komunikasi serial dan tumpukan variabel sementara saat program berjalan. Ilustrasi dari struktur elemen ini dapat dilihat pada **Gambar 4.2**.



Gambar 4.2. Struktur Elemen *Singly Linked List*.

4.3 Struktur Data Pada Memori *Non-Volatile*

Berbeda dengan SRAM, struktur *Linked List* tidak dapat disimpan di dalam EEPROM karena alamat *pointer* akan bernilai acak dan tidak valid setelah daya terputus dan mikrokontroler diaktifkan kembali. Oleh karena itu, data di dalam EEPROM disimpan murni sebagai *Array of Structs*, disimpan secara berurutan.

Untuk menampung seluruh variabel spesifikasi inventaris ke dalam mikrokontroler, tipe data standar ditinggalkan. Sistem menggunakan deklarasi C dengan atribut `__attribute__((packed))` untuk dapat dilakukan pemotongan variabel hingga ke level bit, meniadakan *padding* kosong yang secara umum ditambahkan oleh *compiler*, sehingga struktur ini dapat menghasilkan ukuran 96-bit (12 *byte*) per item. Rincian dari struktur ini dapat dilihat pada **Tabel 4.1** dan **Gambar 4.3**.

Tabel 4.1 Susunan Alokasi Bit pada Struktur *ItemData*.

Nama Variabel	Tipe Data	Ukuran Bit	Representasi
ID	Unsigned Integer	7 bit	Nilai 0-127
pName0 - pName7	Custom 6-Bit Char	48 bit	8 karakter Alfanumerik
category	Unsigned Integer	2 bit	Nilai 0-3
location	Unsigned Integer	6 bit	Nilai 0-63
qtyTersedia	Unsigned Integer	6 bit	Kuantitas 0-63
qtyDipinjam	Unsigned Integer	6 bit	Kuantitas 0-63
qtyRusak	Unsigned Integer	6 bit	Kuantitas 0-63
pic	Custom 5-Bit Char	15 bit	3 karakter <i>Uppercase</i>
Total Kapasitas		96 bit = 12 Byte	

```
struct __attribute__((packed)) ItemData {
```



```
uint8_t id : 7;

uint8_t pName0 : 6;
uint8_t pName1 : 6;
uint8_t pName2 : 6;
uint8_t pName3 : 6;
uint8_t pName4 : 6;
uint8_t pName5 : 6;
uint8_t pName6 : 6;
uint8_t pName7 : 6;

uint8_t category : 2;
uint8_t location : 6;

uint8_t qtyTersedia : 6;
uint8_t qtyDipinjam : 6;
uint8_t qtyRusak : 6;

uint16_t pic : 15;
};
```

Gambar 4.3. Deklarasi atribut pada *database.h*.

4.4 Logika Kompresi dan Pack/Unpack Data Pada Memori Non-Volatile

Variabel yang paling boros memori dalam sistem tertanam adalah *string* berbasis standar ASCII (yaitu 8 bit per karakter). Untuk mencapai ukuran ideal 12 *byte*, karakter teks akan dikompresi secara manual menggunakan sebuah kamus biner desimal khusus (*dictionary encoding*).

Nama barang (maksimal 8 karakter) akan dikonversi menggunakan kamus 6-bit yang memetakan karakter alfabet lengkap beserta angka numerik, memanfaatkan 64 slot biner (2^6). Sementara itu, nama *Person in Charge* (PIC, maksimal 3 karakter) dikompresi dengan ketentuan lebih spesifik, yaitu menggunakan kamus 5-bit yang hanya menyediakan 32 slot biner (2^5), sehingga hanya mendukung huruf *uppercase* murni tanpa angka dan simbol. Pemetaan kompresi nama barang dapat dilihat pada **Tabel 4.2**. Pemetaan kompresi nama dalam bentuk *abbreviation* dari PIC dapat dilihat pada **Tabel 4.3**.

Tabel 4.2. Pemetaan Kampus Kompresi Alfabet Numerik 6-Bit.

Rentang Nilai (desimal)	Format Biner	Representasi Karakter
----------------------------	--------------	-----------------------

0 - 25	000000 - 011001	Huruf Uppercase 'A' hingga 'Z'
26 - 51	011010 - 110011	Huruf Lowercase 'a' hingga 'z'
52 - 61	110100 - 111101	Angka numerik '0' hingga '9'
62 dan 63	111110 & 111111	Karakter Spasi (<i>padding</i>) / NULL

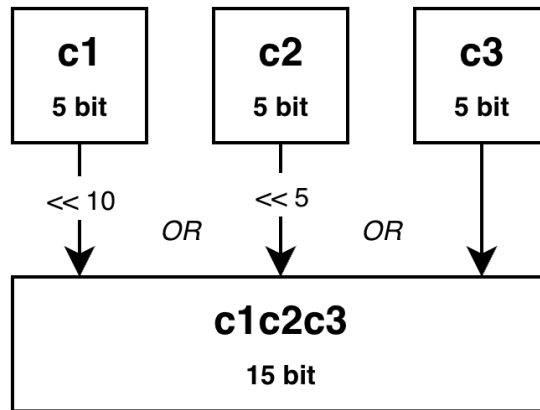
Tabel 4.3. Pemetaan Kampus Kompresi 5-Bit (variabel PIC).

Rentang Nilai (desimal)	Format Biner	Representasi Karakter
0 - 25	00000 - 11001	Huruf Uppercase 'A' hingga 'Z'
26	11010	Karakter Spasi (<i>padding</i>) / NULL
27 - 31	11011 - 11111	Tidak Digunakan

Selain itu, untuk mengimplementasikan kamus kompresi sebelumnya ke dalam memori mikrokontroler, data harus melalui proses translasi mentah saat masuk (proses *Packing*) dan saat keluar (proses *Unpacking*). Karena penggunaan kamus yang spesifik, operasi ini pun dilakukan dengan manipulasi *bitwise* (*Left-Shift*, *Right-Shift*, dan logika *AND/OR*).

1. Proses *Packing* (*compress* ke EEPROM):

Saat mikrokontroler menerima *string* dari antarmuka PC melalui fungsi *packItemData*, *array char* standar (8-bit ASCII) tidak langsung disimpan. Karakter tersebut dilempar ke dalam fungsi *encodeChar5Bit* atau *encodeChar6Bit* untuk diterjemahkan menjadi nilai desimal yang jauh lebih kecil sesuai aturan pada tabel kamus. Khusus untuk variabel PIC, 3 karakter berukuran 5-bit digabung menjadi satu kesatuan memori 15-bit di dalam struktur data. Hal ini dicapai dengan menggeser (*left-shift* <<) bit karakter pertama sebanyak 10 posisi ke kiri, karakter kedua sebanyak 5 posisi ke kiri, dan karakter ketiga tidak digeser. Ketiganya kemudian disatukan menggunakan operasi logika *OR* (|). Ilustrasi proses ini dapat dilihat pada **Gambar 4.4**.

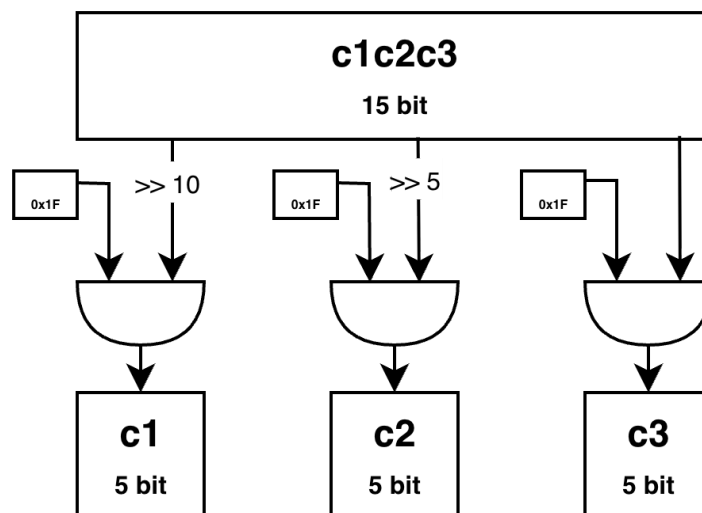


Gambar 4.4. Ilustrasi Operasi Bitwise Packing Variabel PIC.

2. Proses *Unpacking* (*decompress* untuk output UART):

Ketika pengguna dari *client_interface* meminta menampilkan semua data (perintah *GET_ALL*), data masih terkompresi di dalam struktur 12-byte dan harus diekstraksi kembali menjadi *string* C standar agar dapat dicetak ke *Serial Monitor*. Proses ekstraksi ini dilakukan oleh fungsi *unpackItemPIC* dan *unpackItemName*.

Untuk memisahkan kembali gabungan 15-bit nama PIC, sistem menggunakan operasi *Bitmasking*. Sistem akan menggeser blok bit ke kanan (*right-shift* $\gg$) ke posisi semula, lalu menguncinya menggunakan operator *AND* (&) dengan nilai heksadesimal *0x1F* (atau 00011111 dalam biner). *Bitmasking* ini akan memotong sisa bit yang berlebih, menyisakan tepat 5 bit data murni yang kemudian diterjemahkan kembali menjadi karakter ASCII (huruf kapital 'A'-'Z') menggunakan fungsi *decodeChar5Bit*. Ilustrasi proses ini dapat dilihat pada **Gambar 4.5**.



Gambar 4.5. Ilustrasi Operasi *Bitmasking* saat proses *Unpacking*.

4.5 Mekanisme Konsistensi Data Dari Input

Konsistensi data dikelola secara ketat untuk mencegah terjadinya *memory corruption* sebagai akibat dari kegagalan sinkronisasi atau pelanggaran penulisan berlebih. Terdapat dua tahap mekanisme untuk menjaga konsistensi data dari input.

Pertama, sistem secara aktif mencegah terjadinya ketidaksesuaian jumlah item tersimpan saat ini dengan menerapkan penghitungan *real-time*. Modul *database.c* akan menolak permintaan instansiasi *Node* baru dengan pesan galat *ERR: STORAGE_FULL* jika memori telah menyentuh batas *MAX_ITEMS* (85 item), sehingga data tidak akan pernah tertampung di SRAM jika ruang EEPROM aktual tidak mencukupi. Kedua, mekanisme penulisan ke memori *non-volatile* menggunakan metode pembaruan blok, yaitu fungsi *eprom_update_block*. Mikrokontroler akan membandingkan bit di EEPROM dengan bit yang baru datang dari SRAM, dan hanya akan melakukan perubahan memori pada EEPROM jika terdapat perbedaan data.

V. Contoh Studi Kasus (Input-Output) Untuk Masing-Masing Proses dan Analisis Hasil dan Justifikasi Pendekatan Implementasi

Tabel 5.1 Studi Kasus serta Analisis dan Justifikasinya.

Studi Kasus	Analisis Hasil dan Justifikasi
1	Compile program keseluruhan Perintah: <code>./inventory <port arduino></code> Program <code>client_interface_system</code> berjalan dengan sukses: - Program berhasil menangkap argumen input port serial dari terminal, yaitu COM6. - Port COM6 berhasil dibuka, dan program memberikan jeda waktu tunggu (delay) selama 2 detik untuk proses booting mikrokontroler (Arduino). - Setelah koneksi serial siap, program berhasil menampilkan menu utama dengan 7 pilihan menu. - Jeda 2 detik saat booting sangat krusial karena delay ini menjamin bahwa mikrokontroler telah selesai melakukan inisialisasi memori (SRAM/EEPROM) dan siap menerima perintah serial awal.

	<pre> PS C:\Users\TUF\Downloads\EL2008_FinalProject-1\client_interface_windows> ./inventory COM6 ===== SISTEM INVENTARISASI LABORATORIUM PC ===== [INFO] Menggunakan port dari argumen: COM6 [SYSTEM] Mencoba terhubung ke COM6 (Windows)... [SYSTEM] Port terbuka. Menunggu 2 detik untuk Arduino Boot... [SYSTEM] Koneksi Serial Siap! +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): █ </pre>
2	Pilihan Input: di luar range Output: ERROR Eksekusi pilihan menu di luar range berhasil berjalan dengan semestinya: - Program berhasil menerima input angka 10 dari pengguna. - Sistem sukses mendeteksi bahwa angka yang dimasukkan berada di luar jangkauan pilihan yang tersedia (1–7). - Program menampilkan pesan peringatan "Pilihan tidak valid!" sebagai respons terhadap input yang keliru tersebut. - Setelah pesan error dicetak, sistem secara otomatis meminta input menu utama lagi untuk memberikan kesempatan bagi pengguna memasukkan pilihan yang benar. - Mekanisme validasi input terbukti berjalan dengan baik dalam menangani kesalahan pengguna (user error) sehingga mencegah sistem mengeksekusi instruksi ilegal atau mengalami crash.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 10 Pilihan tidak valid! </pre>
3	Tampilkan Semua Input: 5 Output: daftar ID, nama, kategori, lokasi, status, dan PIC barang Eksekusi pilihan menu 5 berhasil berjalan dengan semestinya: - Program berhasil memicu perintah GET_ALL ke arduino dan menerima seluruh data inventaris yang tersimpan di linked list/EEPROM. - Fungsi displayAllInventory sukses mem-parsing string CSV mentah menjadi tampilan tabel terformat rapi. - Angka indeks kategori (0–3) berhasil diterjemahkan menjadi teks string yang telah direpresentasikan. - Setelah tabel selesai dicetak (printTableFooter), sistem secara otomatis meminta input menu utama lagi. - Keberhasilan memunculkan tabel yang memuat semua parameter barang membuktikan bahwa fungsi sistem berjalan baik tanpa mengalami pergeseran data ataupun buffer overflow.

	Pilihan (1-7): 5 <table><tr><th>ID</th><th>Nama</th><th>Kategori</th><th>Lokasi</th><th>Tersedia</th><th>Dipinjam</th><th>Rusak</th><th>PIC</th></tr><tr><td>10</td><td>ESP32</td><td>Mikrokontroler</td><td>12</td><td>2</td><td>0</td><td>0</td><td>AMR</td></tr><tr><td>2</td><td>RASPI</td><td>Mikrokontroler</td><td>2</td><td>2</td><td>1</td><td>0</td><td>MAH</td></tr><tr><td>1</td><td>LIDAR</td><td>Aktuator</td><td>45</td><td>2</td><td>0</td><td>0</td><td>HTN</td></tr><tr><td>5</td><td>LIDAR2</td><td>Aktuator</td><td>45</td><td>1</td><td>1</td><td>0</td><td>HTN</td></tr><tr><td>12</td><td>ESP31</td><td>Aktuator</td><td>12</td><td>10</td><td>2</td><td>0</td><td>YTR</td></tr><tr><td>54</td><td>ESP33</td><td>Aktuator</td><td>24</td><td>5</td><td>0</td><td>0</td><td>AMR</td></tr><tr><td>21</td><td>LIDAR4</td><td>Aktuator</td><td>33</td><td>1</td><td>1</td><td>0</td><td>HTN</td></tr><tr><td>22</td><td>LIDAR5</td><td>Aktuator</td><td>33</td><td>2</td><td>0</td><td>0</td><td>HTH</td></tr></table> SISTEM INVENTARISASI LABORATORIUM 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar Pilihan (1-7): █	ID	Nama	Kategori	Lokasi	Tersedia	Dipinjam	Rusak	PIC	10	ESP32	Mikrokontroler	12	2	0	0	AMR	2	RASPI	Mikrokontroler	2	2	1	0	MAH	1	LIDAR	Aktuator	45	2	0	0	HTN	5	LIDAR2	Aktuator	45	1	1	0	HTN	12	ESP31	Aktuator	12	10	2	0	YTR	54	ESP33	Aktuator	24	5	0	0	AMR	21	LIDAR4	Aktuator	33	1	1	0	HTN	22	LIDAR5	Aktuator	33	2	0	0	HTH
ID	Nama	Kategori	Lokasi	Tersedia	Dipinjam	Rusak	PIC																																																																		
10	ESP32	Mikrokontroler	12	2	0	0	AMR																																																																		
2	RASPI	Mikrokontroler	2	2	1	0	MAH																																																																		
1	LIDAR	Aktuator	45	2	0	0	HTN																																																																		
5	LIDAR2	Aktuator	45	1	1	0	HTN																																																																		
12	ESP31	Aktuator	12	10	2	0	YTR																																																																		
54	ESP33	Aktuator	24	5	0	0	AMR																																																																		
21	LIDAR4	Aktuator	33	1	1	0	HTN																																																																		
22	LIDAR5	Aktuator	33	2	0	0	HTH																																																																		
4	Tambah Data Input: sesuai format Output: loading dan data berhasil ditambahkan, lalu tampilkan semua data. Eksekusi pilihan menu tambah data berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima seluruh parameter masukan barang baru. - Program menampilkan indikator "Loading..." selama proses pengiriman data serial berlangsung ke mikrokontroler. - Pilihan menu 5 (Tampilkan Semua) berhasil membuktikan penambahan data baru tersebut dengan menampilkan objek "Arduino" pada baris terakhir tabel inventaris lengkap dengan kategori "Mikrokontroler". - Keberhasilan memunculkan tabel yang memuat semua parameter barang membuktikan bahwa fungsi sistem berjalan baik tanpa mengalami pergeseran data ataupun buffer overflow. - Masuknya data baru ke posisi paling akhir tabel mengonfirmasi keberhasilan logika alokasi dinamis node pada ujung belakang antrean (tail) di dalam memori SRAM serta penyimpanan permanen di EEPROM.																																																																								

	Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 3 Nama (maks 8 karakter): Arduino Berikut keterangan untuk nomor kategori: 0. Mikrokontroler 1. Aktuator 2. Sensor 3. Lainnya Kategori (0-3): 0 Lokasi (0-63): 1 Qty Tersedia (0-63): 4 Qty Dipinjam (0-63): 3 Qty Rusak (0-63): 0 PIC (3 huruf): CHR Loading... <pre>+-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+</pre> Pilihan (1-7): 5 <table><tr><th>ID</th><th>Nama</th><th>Kategori</th><th>Lokasi</th><th>Tersedia</th><th>Dipinjam</th><th>Rusak</th><th>PIC</th></tr><tr><td>10</td><td>ESP32</td><td>Mikrokontroler</td><td>12</td><td>2</td><td>0</td><td>0</td><td>AMR</td></tr><tr><td>2</td><td>RASPI</td><td>Mikrokontroler</td><td>2</td><td>2</td><td>1</td><td>0</td><td>MAH</td></tr><tr><td>1</td><td>LIDAR</td><td>Aktuator</td><td>45</td><td>2</td><td>0</td><td>0</td><td>HTN</td></tr><tr><td>5</td><td>LIDAR2</td><td>Aktuator</td><td>45</td><td>1</td><td>1</td><td>0</td><td>HTN</td></tr><tr><td>12</td><td>ESP31</td><td>Aktuator</td><td>12</td><td>10</td><td>2</td><td>0</td><td>YTR</td></tr><tr><td>54</td><td>ESP33</td><td>Aktuator</td><td>24</td><td>5</td><td>0</td><td>0</td><td>AMR</td></tr><tr><td>21</td><td>LIDAR4</td><td>Aktuator</td><td>33</td><td>1</td><td>1</td><td>0</td><td>HTN</td></tr><tr><td>22</td><td>LIDAR5</td><td>Aktuator</td><td>33</td><td>2</td><td>0</td><td>0</td><td>HTH</td></tr><tr><td>3</td><td>Arduino</td><td>Mikrokontroler</td><td>1</td><td>4</td><td>3</td><td>0</td><td>CHR</td></tr></table>	ID	Nama	Kategori	Lokasi	Tersedia	Dipinjam	Rusak	PIC	10	ESP32	Mikrokontroler	12	2	0	0	AMR	2	RASPI	Mikrokontroler	2	2	1	0	MAH	1	LIDAR	Aktuator	45	2	0	0	HTN	5	LIDAR2	Aktuator	45	1	1	0	HTN	12	ESP31	Aktuator	12	10	2	0	YTR	54	ESP33	Aktuator	24	5	0	0	AMR	21	LIDAR4	Aktuator	33	1	1	0	HTN	22	LIDAR5	Aktuator	33	2	0	0	HTH	3	Arduino	Mikrokontroler	1	4	3	0	CHR
ID	Nama	Kategori	Lokasi	Tersedia	Dipinjam	Rusak	PIC																																																																										
10	ESP32	Mikrokontroler	12	2	0	0	AMR																																																																										
2	RASPI	Mikrokontroler	2	2	1	0	MAH																																																																										
1	LIDAR	Aktuator	45	2	0	0	HTN																																																																										
5	LIDAR2	Aktuator	45	1	1	0	HTN																																																																										
12	ESP31	Aktuator	12	10	2	0	YTR																																																																										
54	ESP33	Aktuator	24	5	0	0	AMR																																																																										
21	LIDAR4	Aktuator	33	1	1	0	HTN																																																																										
22	LIDAR5	Aktuator	33	2	0	0	HTH																																																																										
3	Arduino	Mikrokontroler	1	4	3	0	CHR																																																																										
5	Tambah Data Input: ID duplicate Output:.ERROR Eksekusi pilihan menu tambah data dengan ID duplikat berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima input ID 1 dari pengguna yang ingin ditambahkan ke dalam basis data inventaris.																																																																																

	- Program mendeteksi adanya kesamaan ID dengan data yang sudah ada (yaitu objek "LIDAR" yang memiliki ID 1), lalu langsung membatalkan proses pendaftaran. - Sistem menampilkan pesan kesalahan "ERROR: ID 1 sudah digunakan!" untuk menginformasikan duplikasi tersebut kepada pengguna.																																																																																
	<table><tr><th>ID</th><th>Nama</th><th>Kategori</th><th>Lokasi</th><th>Tersedia</th><th>Dipinjam</th><th>Rusak</th><th>PIC</th></tr><tr><td>10</td><td>ESP32</td><td>Mikrokontroler</td><td>12</td><td>2</td><td>0</td><td>0</td><td>AMR</td></tr><tr><td>2</td><td>RASPI</td><td>Mikrokontroler</td><td>2</td><td>2</td><td>1</td><td>0</td><td>MAH</td></tr><tr><td>1</td><td>LIDAR</td><td>Aktuator</td><td>45</td><td>2</td><td>0</td><td>0</td><td>HTN</td></tr><tr><td>5</td><td>LIDAR2</td><td>Aktuator</td><td>45</td><td>1</td><td>1</td><td>0</td><td>HTN</td></tr><tr><td>12</td><td>ESP31</td><td>Aktuator</td><td>12</td><td>10</td><td>2</td><td>0</td><td>YTR</td></tr><tr><td>54</td><td>ESP33</td><td>Aktuator</td><td>24</td><td>5</td><td>0</td><td>0</td><td>AMR</td></tr><tr><td>21</td><td>LIDAR4</td><td>Aktuator</td><td>33</td><td>1</td><td>1</td><td>0</td><td>HTN</td></tr><tr><td>22</td><td>LIDAR5</td><td>Aktuator</td><td>33</td><td>2</td><td>0</td><td>0</td><td>HTH</td></tr><tr><td>3</td><td>Arduino</td><td>Mikrokontroler</td><td>1</td><td>4</td><td>3</td><td>0</td><td>CHR</td></tr></table> SISTEM INVENTARISASI LABORATORIUM 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 1 ERROR: ID 1 sudah digunakan!	ID	Nama	Kategori	Lokasi	Tersedia	Dipinjam	Rusak	PIC	10	ESP32	Mikrokontroler	12	2	0	0	AMR	2	RASPI	Mikrokontroler	2	2	1	0	MAH	1	LIDAR	Aktuator	45	2	0	0	HTN	5	LIDAR2	Aktuator	45	1	1	0	HTN	12	ESP31	Aktuator	12	10	2	0	YTR	54	ESP33	Aktuator	24	5	0	0	AMR	21	LIDAR4	Aktuator	33	1	1	0	HTN	22	LIDAR5	Aktuator	33	2	0	0	HTH	3	Arduino	Mikrokontroler	1	4	3	0	CHR
ID	Nama	Kategori	Lokasi	Tersedia	Dipinjam	Rusak	PIC																																																																										
10	ESP32	Mikrokontroler	12	2	0	0	AMR																																																																										
2	RASPI	Mikrokontroler	2	2	1	0	MAH																																																																										
1	LIDAR	Aktuator	45	2	0	0	HTN																																																																										
5	LIDAR2	Aktuator	45	1	1	0	HTN																																																																										
12	ESP31	Aktuator	12	10	2	0	YTR																																																																										
54	ESP33	Aktuator	24	5	0	0	AMR																																																																										
21	LIDAR4	Aktuator	33	1	1	0	HTN																																																																										
22	LIDAR5	Aktuator	33	2	0	0	HTH																																																																										
3	Arduino	Mikrokontroler	1	4	3	0	CHR																																																																										
6	Tambah Data Input: ID diluar range Output: ERROR Eksekusi pilihan menu tambah data dengan ID di luar range berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima input ID 130 dari pengguna pada prompt batasan ID barang yang valid. - Program mendeteksi bahwa nilai ID yang dimasukkan melebihi batas kapasitas maksimum jangkauan yang ditentukan untuk satu data node. - Sistem menampilkan pesan kesalahan "ERROR: ID harus 0-127" untuk memperingatkan pengguna mengenai batasan memori tersebut. - Sistem terbukti mampu menangani masalah ID diluar batas yang sudah ditentukan. - Batasan nilai 0-127 ini memastikan alokasi tipe data integer 8-bit (uint8_t) pada																																																																																

	memori arduino tidak mengalami kegagalan overflow yang dapat merusak struktur data linked list. <pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 130 ERROR: ID harus 0-127 </pre>
7	Tambah Data Input: ID huruf Output: ERROR Eksekusi pilihan menu tambah data dengan ID berupa karakter huruf berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses mendeteksi input karakter huruf "a" pada prompt ID yang seharusnya hanya menerima bilangan bulat (integer). - Program mendeteksi kegagalan konversi tipe data numerik dan mengategorikan input tersebut sebagai nilai yang berada di luar batas aturan jangkauan identifikasi barang. - Sistem menampilkan pesan kesalahan "ERROR: ID harus 0-127" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Penanganan invalid input berupa karakter non-numerik ini berjalan dengan baik.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): a ERROR: ID harus 0-127 </pre>
8	Tambah Data Input: Nama diluar range Output: ERROR Eksekusi pilihan menu tambah data dengan nama di luar range berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima input ID 4 yang valid, kemudian meminta masukan nama barang dengan ketentuan maksimum 8 karakter. - Program mendeteksi input string "osiloskop" yang memiliki panjang 9 karakter, sehingga mengategorikannya sebagai input yang melebihi batas aturan jangkauan nama. - Sistem menampilkan pesan kesalahan "ERROR: Panjang nama harus 1-8 karakter" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Batasan panjang string ini berguna untuk mencegah terjadinya buffer overflow pada array karakter nama di dalam memori arduino.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 4 Nama (maks 8 karakter): osiloskop ERROR: Panjang nama harus 1-8 karakter </pre>
9	Tambah Data Input: nama memiliki karakter bukan huruf atau angka Output: ERROR Eksekusi pilihan menu tambah data dengan nama memiliki karakter bukan huruf atau angka berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima input ID 78 yang valid, kemudian meminta masukan nama barang dengan ketentuan karakter alfanumerik. - Program mendeteksi input string "Budi++" yang mengandung karakter khusus non-alfanumerik (+), sehingga mengategorikannya sebagai input nama yang tidak valid. - Sistem menampilkan pesan kesalahan "ERROR: Nama hanya boleh berisi huruf dan angka" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Sanitasi input terhadap karakter khusus ini berfungsi untuk mencegah kesalahan parsing string berbasis pembatas (delimiter) koma pada protokol komunikasi UART serta menjaga konsistensi penyimpanan data di linked list.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 78 Nama (maks 8 karakter): Budi++ ERROR: Nama hanya boleh berisi huruf dan angka </pre>
10	Tambah Data Input: kategori diluar range Output: ERROR Eksekusi pilihan menu tambah data dengan kategori di luar range berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima parameter ID (4) dan nama (resistor) yang valid, kemudian menampilkan daftar keterangan indeks kategori. - Program mendeteksi input angka kategori 5 dari pengguna yang berada di luar jangkauan aturan batasan indeks yang valid (0-3). - Sistem menampilkan pesan kesalahan "ERROR: Kategori harus 0-3" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Validasi batas nilai kategori ini berfungsi untuk memaksimalkan memori arduino.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 4 Nama (maks 8 karakter): resistor Berikut keterangan untuk nomor kategori: 0. Mikrokontroler 1. Aktuator 2. Sensor 3. Lainnya Kategori (0-3): 5 ERROR: Kategori harus 0-3 </pre>
11	Tambah Data Input: Lokasi diluar range Output: ERROR Eksekusi pilihan menu tambah data dengan lokasi di luar range berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima parameter ID (4), nama (resistor), dan indeks kategori (3) yang valid dari pengguna. - Program mendeteksi input kode lokasi 65 yang berada di luar jangkauan aturan batasan koordinat lokasi yang valid (0–63). - Sistem menampilkan pesan kesalahan "ERROR: Lokasi harus 0-63" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Batasan jangkauan ini berfungsi untuk memaksimalkan memori arduino.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 4 Nama (maks 8 karakter): resistor Berikut keterangan untuk nomor kategori: 0. Mikrokontroler 1. Aktuator 2. Sensor 3. Lainnya Kategori (0-3): 3 Lokasi (0-63): 65 ERROR: Lokasi harus 0-63 </pre>
12	Tambah Data Input: Qty Tersedia diluar range Output: ERROR Eksekusi pilihan menu tambah data dengan Qty Tersedia di luar range berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima parameter ID (4), nama (resistor), indeks kategori (3), dan lokasi (3) yang valid dari pengguna. - Program mendeteksi input jumlah kuantitas tersedia 65 yang berada di luar jangkauan aturan batasan jumlah stok yang valid (0-63). - Sistem menampilkan pesan kesalahan "ERROR: Qty harus 0-63" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Batasan alokasi nilai 0-63 ini berfungsi untuk memaksimalkan memori arduino.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 4 Nama (maks 8 karakter): resistor Berikut keterangan untuk nomor kategori: 0. Mikrokontroler 1. Aktuator 2. Sensor 3. Lainnya Kategori (0-3): 3 Lokasi (0-63): 3 Qty Tersedia (0-63): 65 ERROR: Qty harus 0-63 </pre>
13	Tambah Data Input: Qty Dipinjam diluar range Output: ERROR Eksekusi pilihan menu tambah data dengan Qty Dipinjam di luar range berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima parameter ID (4), nama (resistor), indeks kategori (3), lokasi (3), dan kuantitas tersedia (60) yang valid dari pengguna. - Program mendeteksi input jumlah kuantitas dipinjam 65 yang berada di luar jangkauan aturan batasan jumlah stok yang valid (0–63). - Sistem menampilkan pesan kesalahan "ERROR: Qty harus 0-63" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Batasan nilai 0-63 ini berfungsi untuk memaksimalkan memori arduino.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 4 Nama (maks 8 karakter): resistor Berikut keterangan untuk nomor kategori: 0. Mikrokontroler 1. Aktuator 2. Sensor 3. Lainnya Kategori (0-3): 3 Lokasi (0-63): 3 Qty Tersedia (0-63): 60 Qty Dipinjam (0-63): 65 ERROR: Qty harus 0-63 </pre>
14	Tambah Data Input: Qty Rusak diluar range Output: ERROR Eksekusi pilihan menu tambah data dengan Qty Rusak di luar range berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima parameter ID (4), nama (resistor), indeks kategori (3), lokasi (3), kuantitas tersedia (60), dan kuantitas dipinjam (10) yang valid dari pengguna. - Program mendeteksi input jumlah kuantitas rusak 65 yang berada di luar jangkauan aturan batasan jumlah stok yang valid (0-63). - Sistem menampilkan pesan kesalahan "ERROR: Qty harus 0-63" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Batasan alokasi nilai 0-63 ini berfungsi untuk memaksimalkan memori arduino.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 4 Nama (maks 8 karakter): resistor Berikut keterangan untuk nomor kategori: 0. Mikrokontroler 1. Aktuator 2. Sensor 3. Lainnya Kategori (0-3): 3 Lokasi (0-63): 3 Qty Tersedia (0-63): 60 Qty Dipinjam (0-63): 10 Qty Rusak (0-63): 65 ERROR: Qty harus 0-63 </pre>
15	Tambah Data Input: PIC angka Output: ERROR Eksekusi pilihan menu tambah data dengan PIC berupa angka berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima seluruh parameter numerik dan string nama barang yang valid, hingga sampai pada prompt pengisian identitas penanggung jawab (PIC). - Program mendeteksi input kombinasi angka "123" pada kolom PIC yang memiliki aturan khusus wajib berupa 3 huruf kapital. - Sistem menampilkan pesan kesalahan "ERROR: PIC hanya boleh berisi huruf besar" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Validasi karakter alfabetikal ini berfungsi untuk memastikan kesesuaian kompresi bit array pada memori arduino serta mencegah kerusakan formatting data string saat dipetakan ke dalam struktur linked list.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 4 Nama (maks 8 karakter): resistor Berikut keterangan untuk nomor kategori: 0. Mikrokontroler 1. Aktuator 2. Sensor 3. Lainnya Kategori (0-3): 3 Lokasi (0-63): 3 Qty Tersedia (0-63): 60 Qty Dipinjam (0-63): 10 Qty Rusak (0-63): 5 PIC (3 huruf): 123 ERROR: PIC hanya boleh berisi huruf besar </pre>
16	Tambah Data Input: PIC huruf kecil Output: ERROR Eksekusi pilihan menu tambah data dengan PIC berupa huruf kecil berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima semua parameter numerik dan string dengan benar, hingga pengguna diminta memasukkan inisial penanggung jawab pada prompt PIC. - Program mendeteksi input string "hrl" yang menggunakan huruf kecil, yang mana menyalahi aturan validasi format wajib berupa 3 karakter huruf kapital. - Sistem menampilkan pesan kesalahan "ERROR: PIC hanya boleh berisi huruf besar" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Validasi terhadap casing huruf ini berfungsi untuk memastikan kesesuaian kompresi bit array pada memori arduino serta mencegah inkonsistensi pencarian

	kode data string saat dipetakan ke dalam struktur linked list. <pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 4 Nama (maks 8 karakter): resistor Berikut keterangan untuk nomor kategori: 0. Mikrokontroler 1. Aktuator 2. Sensor 3. Lainnya Kategori (0-3): 3 Lokasi (0-63): 3 Qty Tersedia (0-63): 60 Qty Dipinjam (0-63): 10 Qty Rusak (0-63): 5 PIC (3 huruf): hrl ERROR: PIC hanya boleh berisi huruf besar </pre>
17	Tambah Data Input: PIC di luar 3 huruf Output: ERROR Eksekusi pilihan menu tambah data dengan PIC di luar 3 huruf berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima seluruh parameter numerik dan string dengan benar, hingga pengguna diminta memasukkan inisial penanggung jawab pada prompt PIC. - Program mendeteksi input string "HUFNAGEL" yang memiliki panjang 8 karakter, yang mana melanggar batas ketentuan aturan wajib yaitu tepat berisi 3 karakter huruf. - Sistem menampilkan pesan kesalahan "ERROR: PIC harus tepat terdiri dari 3

	karakter" untuk menghentikan proses pengisian parameter barang lebih lanjut. - Validasi panjang karakter alfanumerik ini berfungsi untuk memaksimalkan memori arduino. <pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 4 Nama (maks 8 karakter): resistor Berikut keterangan untuk nomor kategori: 0. Mikrokontroler 1. Aktuator 2. Sensor 3. Lainnya Kategori (0-3): 3 Lokasi (0-63): 3 Qty Tersedia (0-63): 60 Qty Dipinjam (0-63): 10 Qty Rusak (0-63): 5 PIC (3 huruf): HUFNAGEL ERROR: PIC harus tepat terdiri dari 3 karakter </pre>
18	Tambah data Jumlah item dalam inventaris sudah 85 Output: ERROR Eksekusi pilihan menu tambah data saat jumlah item dalam inventaris sudah mencapai batas maksimum berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka 1 untuk masuk ke dalam menu pencatatan "TAMBAH INVENTARIS". - Sistem sukses menerima input ID 45 dari pengguna pada batasan awal pengisian objek barang baru. - Program melakukan pengecekan kapasitas total basis data dan mendeteksi bahwa kuantitas item aktif yang tersimpan di dalam memori telah menyentuh batas kritis (85 item). - Sistem membatalkan penambahan objek baru dan menampilkan pesan kesalahan

	"[GAGAL] Kapasitas penyimpanan EEPROM penuh (Maks 85 item)!" beserta solusi operasional kepada pengguna. - Mekanisme pembatasan range pada parameter-parameter input berfungsi untuk mencegah terjadinya stack/heap collision akibat alokasi dinamis linked list yang berlebih pada SRAM serta menjaga agar penyimpanan data serial tidak melewati batas memori EEPROM arduino. <pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 1 ===== TAMBAH INVENTARIS ===== ID (0-127): 45 ===== [GAGAL] Kapasitas penyimpanan EEPROM penuh (Maks 85 item)! [INFO] Silakan hapus item lain terlebih dahulu untuk menambahkan data inventaris terbaru. ===== </pre>
19	Hapus Data Input: sesuai format Output: konfirmasi data berhasil dihapus Eksekusi pilihan menu hapus data berhasil berjalan dengan semestinya: - Program berhasil memproses input untuk masuk ke dalam menu "HAPUS INVENTARIS". - Sistem sukses menerima parameter ID dari pengguna. - Program memverifikasi ID tersebut dengan mengambil data terbaru (GET_ALL) dari mikrokontroler dan berhasil menemukan ID yang cocok di dalam basis data. - Program menampilkan indikator "Loading..." selama proses pengiriman perintah serial DEL ke Arduino berlangsung. - Sistem menampilkan pesan "ID [X] berhasil dihapus!" - Keberhasilan ini membuktikan bahwa instruksi penghapusan telah diterima dengan baik, node target sukses dilepaskan (free) dari linked list di dalam memori SRAM, dan pembaruan memori EEPROM telah tersinkronisasi.

	<pre> +-----+ ID Nama Kategori Lokasi Tersedia Dipinjam Rusak PIC +-----+ 10 ESP32 Mikrokontroler 12 2 0 0 AMR 2 RASPI Mikrokontroler 2 2 1 0 MAH 1 LIDAR Aktuator 45 2 0 0 HTN 5 LIDAR2 Aktuator 45 1 1 0 HTN 12 ESP31 Aktuator 12 10 2 0 YTR 54 ESP33 Aktuator 24 5 0 0 AMR 21 LIDAR4 Aktuator 33 1 1 0 HTN 22 LIDAR5 Aktuator 33 2 0 0 HTH 3 Arduino Mikrokontroler 1 4 3 0 CHR +-----+ +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 2 ===== HAPUS INVENTARIS ===== Masukkan ID: 3 Loading... ID 3 berhasil dihapus! +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 5 +-----+ ID Nama Kategori Lokasi Tersedia Dipinjam Rusak PIC +-----+ 10 ESP32 Mikrokontroler 12 2 0 0 AMR 2 RASPI Mikrokontroler 2 2 1 0 MAH 1 LIDAR Aktuator 45 2 0 0 HTN 5 LIDAR2 Aktuator 45 1 1 0 HTN 12 ESP31 Aktuator 12 10 2 0 YTR 54 ESP33 Aktuator 24 5 0 0 AMR 21 LIDAR4 Aktuator 33 1 1 0 HTN 22 LIDAR5 Aktuator 33 2 0 0 HTH +-----+ </pre>	
	Hapus Data Input: ID tidak dalam database Output: ERROR Eksekusi pilihan menu hapus data dengan ID yang tidak terdaftar berhasil berjalan dengan semestinya: - Program berhasil memproses input untuk masuk ke dalam menu "HAPUS	20

	INVENTARIS". - Sistem menerima input ID dari pengguna untuk dihapus. - Program melakukan pengecekan ke dalam basis data melalui pembacaan buffer serial dan mendeteksi bahwa ID tersebut tidak eksis di dalam sistem. - Program secara otomatis membatalkan pengiriman perintah DEL ke mikrokontroler untuk mencegah modifikasi memori yang tidak valid. - Sistem menampilkan pesan "ERROR: ID [X] tidak ditemukan!" - Penanganan apabila tidak ditemukannya ID dalam database berjalan dengan baik. <pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 2 ===== HAPUS INVENTARIS ===== Masukkan ID: 99 Loading... ERROR: ID 99 tidak ditemukan! </pre>
21	Cari data (ID) Input: format sesuai Output: menampilkan informasi detail barang yang dicari. Eksekusi pilihan menu pencarian data berdasarkan ID berhasil berjalan dengan semestinya: - Program berhasil memproses input untuk masuk ke dalam menu pencarian barang. - Sistem menerima input ID target dari pengguna secara valid. - Program mengambil seluruh daftar inventaris dari mikrokontroler dan melakukan iterasi pencarian ID (parsing) pada buffer serial. - Program berhasil menemukan ID yang cocok dan memisahkan (parsing)

	parameter datanya (Nama, Kategori, Lokasi, Kuantitas, PIC). - Keberhasilan eksekusi dibuktikan dengan ditampilkan baris informasi tunggal atau tabel yang secara presisi hanya memuat detail barang dari ID yang dicari, membuktikan logika pemotongan string (strtok) bekerja dengan sempurna.																																																																
	<table><tr><th>ID</th><th>Nama</th><th>Kategori</th><th>Lokasi</th><th>Tersedia</th><th>Dipinjam</th><th>Rusak</th><th>PIC</th></tr><tr><td>2</td><td>RASPI</td><td>Mikrokontroler</td><td>2</td><td>2</td><td>1</td><td>0</td><td>MAH</td></tr><tr><td>1</td><td>LIDAR</td><td>Aktuator</td><td>45</td><td>2</td><td>0</td><td>0</td><td>HTN</td></tr><tr><td>5</td><td>LIDAR2</td><td>Aktuator</td><td>45</td><td>1</td><td>1</td><td>0</td><td>HTN</td></tr><tr><td>12</td><td>ESP31</td><td>Aktuator</td><td>12</td><td>10</td><td>2</td><td>0</td><td>YTR</td></tr><tr><td>54</td><td>ESP33</td><td>Aktuator</td><td>24</td><td>5</td><td>0</td><td>0</td><td>AMR</td></tr><tr><td>21</td><td>LIDAR4</td><td>Aktuator</td><td>33</td><td>1</td><td>1</td><td>0</td><td>HTN</td></tr><tr><td>22</td><td>LIDAR5</td><td>Aktuator</td><td>33</td><td>2</td><td>0</td><td>0</td><td>HTH</td></tr></table> SISTEM INVENTARISASI LABORATORIUM 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar Pilihan (1-7): 3 ===== CARI BERDASARKAN ID ===== Masukkan ID: 2 ===== DATA DITEMUKAN ===== ID : 2 Nama : RASPI Kategori : Mikrokontroler Lokasi : 2 Tersedia : 2 Dipinjam : 1 Rusak : 0 PIC : MAH	ID	Nama	Kategori	Lokasi	Tersedia	Dipinjam	Rusak	PIC	2	RASPI	Mikrokontroler	2	2	1	0	MAH	1	LIDAR	Aktuator	45	2	0	0	HTN	5	LIDAR2	Aktuator	45	1	1	0	HTN	12	ESP31	Aktuator	12	10	2	0	YTR	54	ESP33	Aktuator	24	5	0	0	AMR	21	LIDAR4	Aktuator	33	1	1	0	HTN	22	LIDAR5	Aktuator	33	2	0	0	HTH
ID	Nama	Kategori	Lokasi	Tersedia	Dipinjam	Rusak	PIC																																																										
2	RASPI	Mikrokontroler	2	2	1	0	MAH																																																										
1	LIDAR	Aktuator	45	2	0	0	HTN																																																										
5	LIDAR2	Aktuator	45	1	1	0	HTN																																																										
12	ESP31	Aktuator	12	10	2	0	YTR																																																										
54	ESP33	Aktuator	24	5	0	0	AMR																																																										
21	LIDAR4	Aktuator	33	1	1	0	HTN																																																										
22	LIDAR5	Aktuator	33	2	0	0	HTH																																																										
22	Cari Data (ID) Input: ID tidak dalam database Output: ERROR Eksekusi pilihan menu pencarian data dengan ID yang tidak terdaftar berhasil berjalan dengan semestinya: - Program berhasil memproses input untuk masuk ke dalam menu pencarian. - Sistem menerima masukan target ID dari pengguna. - Program melakukan iterasi pencarian (parsing) baris demi baris pada buffer serial hingga menemukan batas akhir pengiriman data (END). - Karena logika if pengecekan ID tidak pernah terpenuhi hingga iterasi selesai,																																																																

	program mengonfirmasi absennya data tersebut. - Sistem memunculkan pesan peringatan atau ERROR yang menyatakan bahwa ID tidak ditemukan, mengonfirmasi bahwa program tidak crash atau menampilkan data sampah (garbage value) ketika referensi memori tidak ada. <pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 3 ===== CARI BERDASARKAN ID ===== Masukkan ID: 99 ID tidak ditemukan.</pre>
23	Update Stok & Status Input: sesuai format Output: loading dan data berhasil di-update. Eksekusi pilihan menu update stok & status berhasil berjalan dengan semestinya: - Program berhasil memproses input untuk masuk ke dalam menu "UPDATE INVENTARIS". - Sistem sukses memverifikasi keberadaan ID barang di dalam basis data melalui komparasi ID pada buffer serial. - Program berhasil menerima semua masukan nilai baru secara valid (Qty Tersedia, Qty Dipinjam, dan Qty Rusak). - Program menampilkan indikator "Loading..." selama pengiriman perintah UPDATE dengan format string beruntun (koma) ke mikrokontroler. - Perubahan status ini dapat dibuktikan melalui menu "Tampilkan Semua", di mana kuantitas barang dengan ID tersebut telah berubah secara tepat tanpa mengubah parameter statis lainnya (seperti Nama atau PIC). - Hal ini mengonfirmasi bahwa penimpaan (overwriting) data pada node target di SRAM dan EEPROM telah dieksekusi di blok memori yang benar.

===== CARI BERDASARKAN ID =====
Masukkan ID: 2

===== DATA DITEMUKAN =====
ID : 2
Nama : RASPI
Kategori : Mikrokontroler
Lokasi : 2
Tersedia : 3
Dipinjam : 2
Rusak : 0
PIC : MAH

```
+-----+
|      SISTEM INVENTARISASI LABORATORIUM      |
+-----+
| 1. Tambah Data                               |
| 2. Hapus Data                               |
| 3. Cari Data (ID)                           |
| 4. Update Stok & Status                     |
| 5. Tampilkan Semua                          |
| 6. Ringkasan Inventaris                     |
| 7. Keluar                                   |
+-----+
```

Pilihan (1-7): 4

===== UPDATE INVENTARIS =====
ID Barang: 2
Qty Tersedia Baru: 1
Qty Dipinjam Baru: 4
Qty Rusak Baru: 0

Loading...

Stock dan status berhasil diperbarui!

```
+-----+
|      SISTEM INVENTARISASI LABORATORIUM      |
+-----+
| 1. Tambah Data                               |
| 2. Hapus Data                               |
| 3. Cari Data (ID)                           |
| 4. Update Stok & Status                     |
| 5. Tampilkan Semua                          |
| 6. Ringkasan Inventaris                     |
| 7. Keluar                                   |
+-----+
```

Pilihan (1-7): 3

===== CARI BERDASARKAN ID =====
Masukkan ID: 2

===== DATA DITEMUKAN =====
ID : 2
Nama : RASPI
Kategori : Mikrokontroler
Lokasi : 2
Tersedia : 1
Dipinjam : 4
Rusak : 0
PIC : MAH

24	Update Stok & Status Input: ID tidak dalam database Output: ERROR Eksekusi pilihan menu update stok & status dengan ID yang tidak terdaftar berhasil berjalan dengan semestinya: - Program masuk ke dalam fase modifikasi melalui menu "UPDATE INVENTARIS". - Sistem menerima input ID yang ingin diubah datanya. - Logika validasi program berjalan sebelum meminta input kuantitas baru; program mendeteksi bahwa ID tidak tersedia dalam sistem. - Sistem langsung membatalkan eksekusi, menolak meminta parameter Qty lebih lanjut, dan tidak mengirimkan perintah UART apa pun ke Arduino. - Program menampilkan "ERROR: ID [X] tidak ditemukan!", membuktikan bahwa sistem memiliki proteksi terhadap perintah pembaruan fiktif (blind update). <pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 4 ===== UPDATE INVENTARIS ===== ID Barang: 99 ERROR: ID 99 tidak ditemukan! </pre>
25	Update Stok & Status Input: qTer, qDip, atau qRus tidak di range 0-63 Output: ERROR Eksekusi pilihan menu update stok & status dengan input kuantitas di luar range berhasil berjalan dengan semestinya: - Program berhasil memverifikasi ID yang valid dan melanjutkan ke tahap penerimaan parameter kuantitas baru.

	- Pengguna memasukkan salah satu atau beberapa angka di luar rentang valid (misal: negatif atau lebih dari 63). - Logika perlindungan batasan (boundary check) mendeteksi anomali pada nilai qTer, qDip, atau qRus. - Program secara instan membatalkan konstruksi string perintah serial untuk mencegah masuknya nilai yang dapat menyebabkan kerusakan data (data corruption) atau overflow pada tipe data penyimpanan mikrokontroler. - Sistem menampilkan pesan "ERROR: Qty harus 0-63", mengembalikan pengguna dengan aman keluar dari fungsi tersebut. <pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 4 ===== UPDATE INVENTARIS ===== ID Barang: 2 Qty Tersedia (0-63): 64 ERROR: Qty harus 0-63 </pre>
26	Ringkasan Inventaris Input: 6 Output: menampilkan ringkasan inventaris Eksekusi pilihan menu ringkasan inventaris berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan untuk masuk ke dalam menu "RINGKASAN INVENTARIS". - Sistem melakukan perintah pengambilan data secara menyeluruh (GET_ALL) ke mikrokontroler. - Program berhasil melakukan iterasi pembacaan (parsing) pada seluruh baris data di buffer serial untuk mengakumulasi nilai-nilai kuantitas (total barang tersedia, dipinjam, dan rusak) serta mengelompokkan data berdasarkan kategori atau lokasi.

	- Sistem memunculkan hasil rekapitulasi data (statistik) dengan format teks atau tabel yang terstruktur di layar. - Program menyelesaikan perhitungan tanpa mengalami crash, salah hitung (miscalculation), atau segmentation fault, membuktikan bahwa algoritma akumulasi nilai integer dari variabel string buffer berjalan dengan presisi. <pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 6 ===== RINGKASAN ===== Jumlah Jenis Barang : 7 Total Tersedia : 21 Total Dipinjam : 5 Total Rusak : 0 ===== </pre>
27	Keluar Input: 7 Output: program berhenti dengan aman (graceful exit). Eksekusi pilihan menu keluar program berhasil berjalan dengan semestinya: - Program berhasil memproses input pilihan angka untuk perintah terminasi atau "KELUAR". - Sistem mengeksekusi prosedur penutupan (clean-up), termasuk menutup akses (men-close handle) pada port serial (COM) yang terhubung ke mikrokontroler secara aman untuk mencegah status port tertahan (port hanging) di sistem operasi. - Program berhasil memutus perulangan utama (main loop) dan terminasi dijalankan tanpa memunculkan pesan error atau menyebabkan memory leak. - Sistem mengembalikan status exit code 0 (sukses) dan menyerahkan kembali kontrol penuh kepada terminal Windows, menandakan bahwa siklus hidup program C telah selesai dengan sempurna.

	<pre> +-----+ SISTEM INVENTARISASI LABORATORIUM +-----+ 1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar +-----+ Pilihan (1-7): 7 Keluar dari program... [SYSTEM] Koneksi Serial Windows diputus. </pre>
--	---

VI. Pembagian Kerja Kelompok Secara Substansial Berdasarkan Dekomposisi Permasalahan

6.1 Pembagian Tugas Pengerjaan Laporan dan Slide Presentasi

Tabel 6.1.1 Pembagian Tugas Pengerjaan Laporan dan Slide Presentasi.

PEMBAGIAN TUGAS		
No	Komponen Tugas	PIC
A. LAPORAN		
1	Deskripsi Permasalahan	Nessa (13224078)
2	Dekomposisi & Solusi Permasalahan	Nessa (13224078)
3	Struktur Data & Logika	Hanif (13224087)
4	Flowchart	<i>terdapat di list flowchart</i>
5	DFD	Hanif (13224087)
6	Studi Kasus	Chrisjo and nagel

7	Analisis & Justifikasi	Chrisjo and nagel
8	Analisis Keterbatasan Sistem	Hanif (13224087)
9	Pembagian Tugas	<i>bersama</i>
B. Slides Presentasi		
1	Deskripsi & Dekomposisi Masalah	Nessa (13224078)
2	Deskripsi Sistem	Nessa (13224078)
3	Arsitektur Memori Arduino	Nessa (13224078)
4	Studi Kasus	Nessa (13224078)
5	Keterbatasan Sistem	Nessa (13224078)
6	Kesimpulan & Analisis Hasil	Nessa (13224078)
7	Demonstrasi	<i>bersama</i>
8	Pembagian Kerja & Kontribusi	

6.2 Pembagian Tugas Pengerjaan Flowchart

Tabel 6.1.2 Pembagian Tugas Pengerjaan Flowchart.

Flowchart		
No	Nama Fungsi	PIC
app_logic.c		
1	runApp	Nagel (13224015)
2	addInventory	Nagel (13224015)
3	deleteInventory	Nagel (13224015)
4	searchByID	Nagel (13224015)
5	updateStockAndStatus	Chrisjo (13224108)
6	displayAllInventory	Chrisjo (13224108)

7	displaySummary	Chrisjo (13224108)
8	fetchSerialUntil	Nessa (13224078)
database.c		
1	initUART	Nessa (13224078)
2	printString	Chrisjo (13224108)
3	initDatabase	Nessa (13224078)
4	processSerialInput	Nagel (13224015)
5	encodeChar6Bit	Nessa (13224078)
6	decodeChar6Bit	Nessa (13224078)
7	encodeChar5Bit	Nessa (13224078)
8	decodeChar5Bit	Nessa (13224078)
9	packItemData	Nessa (13224078)
10	unpackItemName	Nessa (13224078)
11	unpackItemPIC	Nessa (13224078)
12	insertItem	Chrisjo (13224108)
13	deleteItem	Chrisjo (13224108)
14	updateItem	Chrisjo (13224108)
15	syncListToEEPROM	Chrisjo (13224108)
16	printInventory	Chrisjo (13224108)
17	parseAndAddCommand	Chrisjo (13224108)
18	dataAvailableUART	Chrisjo (13224108)
19	receiveUART	Chrisjo (13224108)
20	transmitUART	Chrisjo (13224108)
21	printInt	Nessa (13224078)
serial_comm		
1	initSerialPort	Hanif (13224087)
2	sendSerialData	Hanif (13224087)
3	receiveSerialData	Hanif (13224087)
4	closeSerialPort	Hanif (13224087)
user_interface (DONE)		

1	printMainMenu	Nessa (13224078)
2	getUserChoice	Nessa (13224078)
3	promptString	Nessa (13224078)
4	promptInt	Nessa (13224078)
5	printTableHeader	Nessa (13224078)
6	printTableFooter	Nessa (13224078)

6.3 Pembagian Tugas Pembuatan Program C

Tabel 6.1.3 Pembagian Tugas Pembuatan Program C.

Pembuatan Program C		
No	Nama Fungsi	PIC
app_logic.c		
1	runApp	Nagel (13224015)
2	addInventory	Nagel (13224015)
3	deleteInventory	Nagel (13224015)
4	searchByID	Nagel (13224015)
5	updateStockAndStatus	Chrisjo (13224108)
6	displayAllInventory	Chrisjo (13224108)
7	displaySummary	Chrisjo (13224108)
8	fetchSerialUntil	Chrisjo (13224108)
database.c		
1	initUART	Hanif (13224087)
2	printString	Chrisjo (13224108)
3	initDatabase	Hanif (13224087)
4	processSerialInput	Chrisjo (13224108)
5	encodeChar6Bit	Hanif (13224087)

6	decodeChar6Bit	Hanif (13224087)
7	encodeChar5Bit	Hanif (13224087)
8	decodeChar5Bit	Hanif (13224087)
9	packItemData	Hanif (13224087)
10	unpackItemName	Hanif (13224087)
11	unpackItemPIC	Hanif (13224087)
12	insertItem	Chrisjo (13224108)
13	deleteItem	Chrisjo (13224108)
14	updateItem	Chrisjo (13224108)
15	syncListToEEPROM	Hanif (13224087)
16	printInventory	Chrisjo (13224108)
17	parseAndAddCommand	Chrisjo (13224108)
18	dataAvailableUART	Hanif (13224087)
19	receiveUART	Hanif (13224087)
20	transmitUART	Hanif (13224087)
serial_comm		
1	initSerialPort	Hanif (13224087)
2	sendSerialData	Hanif (13224087)
3	receiveSerialData	Hanif (13224087)
4	closeSerialPort	Hanif (13224087)
user_interface		
1	printMainMenu	Nessa (13224078)
2	getUserChoice	Nessa (13224078)
3	promptString	Nessa (13224078)
4	promptInt	Nessa (13224078)
5	printTableHeader	Nessa (13224078)
6	printTableFooter	Nessa (13224078)

VII. Analisis Keterbatasan Sistem

7.1 Keterbatasan Kapasitas Memori Non-Volatile

Keterbatasan perangkat keras yang paling absolut dalam arsitektur basis data ini terletak pada kapasitas memori *non-volatile* mikrokontroler Arduino UNO, yang hanya memiliki EEPROM sebesar 1 KB (1.024 *byte*). Walau kami sudah mengimplementasikan sistem dengan kompresi data tingkat bit (*bit-level packing*) untuk mereduksi ukuran setiap entitas inventaris menjadi tepat 12 *byte*, kapasitas

penyimpanan tetap memiliki batas sebanyak 85 item unik. Batas ini ditetapkan secara definitif di dalam kode melalui variabel makro *MAX_ITEMS* untuk mengakomodasi struktur data serta menyisakan sedikit memori untuk terminasi basis data. Sebagai dampaknya, sistem ini tidak dapat dimanfaatkan untuk inventarisasi skala menengah atau besar tanpa adanya integrasi modul memori eksternal seperti *I2C EEPROM*. Untuk mencegah terjadinya korupsi memori atau ketidaksesuaian antara data SRAM dan EEPROM akibat penulisan yang melampaui batas alamat memori fisik, program telah kami lengkapi dengan proteksi *real-time*. Jika pengguna berupaya memasukkan item ke-86, sistem akan memblokir instansiasi *Linked List* di SRAM, menggagalkan sinkronisasi EEPROM, dan secara otomatis mengirimkan informasi *ERR: STORAGE_FULL* ke program *client_interface*.

7.2 Restriksi Format Karakter dan Pemotongan String

Berdasarkan rincian pada **Bab IV**, kami melakukan implementasi algoritma dengan konsep *dictionary encoding* untuk dapat mencapai ukuran struktur memori 12 *byte*. Terlihat efisien, namun hal ini menimbulkan konsekuensi langsung terhadap fleksibilitas format teks yang dapat disimpan oleh sistem. Pemetaan memori 6-bit untuk variabel nama barang hanya mengizinkan 64 kombinasi karakter, yang dialokasikan secara murni untuk alfabet kapital ('A'-'Z'), alfabet kecil ('a'-'z'), angka numerik ('0'-'9'), dan spasi. Konsekuensi utama dari sistem ini secara arsitektural adalah tidak memiliki dukungan terhadap karakter khusus atau simbol baca (seperti tanda hubung, garis bawah, atau tanda kurung). Hal yang sama berlaku pada variabel nama penanggung jawab (PIC) yang dikompresi menjadi 5-bit, sehingga hanya mampu menyimpan 32 kombinasi karakter yang dibatasi pada alfabet kapital ('A'-'Z') dan spasi saja. Selain itu, keterbatasan memori juga menyebabkan penggunaan jumlah maksimal karakter, yaitu nama barang dibatasi hanya 8 karakter, dan nama PIC dibatasi hanya 3 karakter. Segala bentuk *string* yang melebihi batas ini atau mengandung simbol yang tidak dipetakan akan divalidasi terlebih dahulu oleh antarmuka klien sebelum dikirim, agar mikrokontroler tidak mengalami kegagalan *parsing* atau menyimpan *garbage data*.

7.3 Batas Kuantitas Barang dan Rentang ID

Penggunaan deklarasi *C bit-fields* (`__attribute__((packed))`) untuk dapat membuang *padding* memori menciptakan sebuah batasan pada nilai numerik maksimum yang dapat direpresentasikan oleh variabel. Kuantitas barang (terdiri dari status Tersedia, Dipinjam, dan Rusak) masing-masing dialokasikan ruang sebesar 6 bit. Berdasarkan batasan biner dasar, nilai desimal maksimal yang dapat ditampung dalam 6 bit adalah $2^6 - 1 = 63$. Secara paralel, variabel ID (nomor identitas unik) dialokasikan ruang 7 bit, yang membatasi rentang penomoran dari 0 hingga 127 (batas maksimal: $2^7 - 1 = 127$). Keterbatasan ini mengharuskan adanya adaptasi prosedur operasional di laboratorium. Apabila terdapat satu jenis barang, contohnya komponen Resistor 1K yang jumlah fisiknya melebihi 63 unit,

sistem tidak dapat mencatatnya di dalam satu baris data (satu kali input). Barang tersebut harus dipecah atau diinput menggunakan dua ID yang berbeda (misal: ID 1 untuk 60 unit pertama, dan ID 2 untuk sisa unitnya).

7.4 Delay Pada Operasi I/O

Dari aspek manajemen kinerja dari sistem kami, arsitektur *Linked List* dinamis di dalam SRAM (2 KB) menimbulkan sebuah tantangan krusial. Tantangan ini adalah meskipun pembaruan data ke EEPROM sudah kami coba optimalkan dengan penggunaan fungsi *eeeprom_update_block* untuk memotong waktu tulis (sebelumnya saat tahap pengembangan, kami sempat menggunakan *eeeprom_write_block*), operasi penghapusan data (*Delete*) pada pertengahan atau awal tumpukan *Linked List* tetap memicu kompleksitas waktu yang menyebabkan *delay* pada operasi I/O. Mikrokontroler harus melakukan sinkronisasi massal untuk menggeser struktur data, yang memakan waktu beberapa milidetik hingga detik (tergantung pada jumlah *node* yang perlu diubah).

7.5 Ketergantungan Terhadap Proses Validasi *Client Interface* (PC)

Untuk meminimalisasi jejak memori (*memory footprint*) dari ukuran kompilasi *firmware* (*Flash Memory*) dan SRAM, file *database.c* kami rancang dengan sifat utama sebagai mesin basis data pasif (*passive database engine*). Hal ini berarti mikrokontroler tidak dilengkapi dengan logika validasi input, pembersihan *string*, atau mekanisme perlindungan *buffer overflow* yang kompleks dan ideal. Akibatnya, sistem ini tidak dapat berdiri sendiri (*standalone*) sebagai sistem yang kebal terhadap kesalahan input jika diakses secara manual melalui Serial Monitor bawaan Arduino IDE. Sistem bergantung pada program antarmuka klien berbasis C di PC (*Client Interface*) untuk menyaring, memvalidasi panjang, dan memformat paket data serial menjadi perintah baku (misal: `"ADD,1,LIDAR,0,12,2,0,0,AMR\n"`).

VIII. Kesimpulan dan Saran

8.1 Kesimpulan

Berdasarkan perancangan dan pengujian dari sistem keseluruhan yang telah dilakukan, dapat disimpulkan bahwa Sistem Inventarisasi Laboratorium berbasis arsitektur *Client-Server* berhasil diimplementasikan secara efisien untuk mengatasi keterbatasan sumber daya *memory* mikrokontroler Arduino UNO. Melalui pendekatan low-level, yaitu memadukan kompresi data hingga pengaturan level bit dan sinkronisasi memori secara berkala antara SRAM dan EEPROM, sistem ini terbukti mampu mengelola basis data secara stabil, responsif, dan aman dari risiko kelebihan beban kapasitas. Berikut adalah kesimpulan utama dari pengembangan tugas besar kami:

- **Implementasi Arsitektur *Dual-Memory*:** Sistem kami dapat mengintegrasikan SRAM untuk manipulasi data dinamis yang cepat

menggunakan *Singly Linked List*, dan EEPROM sebagai penyimpanan *non-volatile* untuk dapat menyimpan data walau daya Arduino UNO diputus.

- **Kompresi Data Hingga Bit Level:** Penggunaan manipulasi *bit-field* serta fungsi *custom encoder-decoder* (6-bit untuk karakter alfanumerik dan 5-bit untuk inisial PIC) berhasil mendukung ukuran satu struktur data (*struct*) item secara maksimal menjadi hanya 12 byte.
- **Optimalisasi Kapasitas Penyimpanan Maksimal:** Karena proses kompresi sebelumnya dan digunakan beberapa batasan input data secara spesifik, sistem kami mampu menampung kapasitas absolut hingga 85 barang di dalam memori EEPROM berukuran 1.024 byte.
- **Mekanisme Konsistensi Data:** Program antarmuka PC dapat kami desain untuk bekerja sebagai tempat validasi tipe *input* data yang sesuai, serta mampu menghitung beban memori secara *real-time* dan memblokir *input* berlebih (jika sudah menyimpan tepat 85 item) sebelum dikirim ke mikrokontroler.

8.2 Saran

Untuk pengembangan sistem inventarisasi untuk Laboratorium Embedded Systems ke depannya, beberapa saran dan langkah adaptif berikut dapat dipertimbangkan untuk diimplementasikan:

- **Pengembangan Antarmuka Grafis (GUI):** Hal ini artinya mengembangkan aplikasi *Client* PC dari yang saat ini berbasis antarmuka baris perintah (CLI/Terminal) menjadi aplikasi *desktop* visual atau *dashboard* berbasis web yang lebih ramah pengguna (*user-friendly*) bagi laboran yang mengoperasikannya.
- **Validasi Lebih Lanjut pada Sistem Database Arduino UNO:** Konsep ini artinya selain validasi yang dilakukan pada *client interface*, dilakukan juga validasi data pada program *database* Arduino UNO, sehingga tidak akan terjadi kesalahan struktur data jika dilakukan input selain melalui *client interface* yang sudah dibuat khusus untuk program ini.
- **Penambahan Lapisan Keamanan dan Autentikasi:** Konsep ini artinya menerapkan fitur otorisasi (misal: menggunakan pemindai RFID KTM mahasiswa atau *password* admin) agar operasi yang bersifat kritis seperti penambahan data, penghapusan data, dan manipulasi stok rusak tidak dapat dilakukan oleh pihak yang tidak berwenang.
- **Integrasi Pemindai QR Code Aktual:** Konsep ini artinya mengembangkan sistem lebih lanjut dengan mengintegrasikan modul pembaca QR code perangkat keras langsung ke mikrokontroler, sehingga proses identifikasi ID barang saat peminjaman menjadi sepenuhnya otomatis sesuai dengan deskripsi masalah awal.

IX. Sumber

[1] avrdudes, “GitHub - avrdudes/avrdude: AVRDUDE is a utility to program AVR microcontrollers,” *GitHub*, Jun. 23, 2025. <https://github.com/avrdudes/avrdude/> (accessed Jun, 2026).

X. Lampiran

Lampiran 1. Link Slides Presentasi.

[Slide Presentasi Tugas Besar PMP Kelompok 6 K03](#)

Lampiran 2. Link Video Pemaparan Slides Presentasi dan Demonstrasi.

[Video Presentasi Tugas Besar PMP Kelompok 6 K03](#)

Lampiran 3. Link Repository Github Tugas Besar EL2008 K03 Kelompok 6.

https://github.com/hanif-page/EL2008_FinalProject